



JAL2501 MACHINE LEARNING

UNIT II SUPERVISED LEARNING

UNIT II SUPERVISED LEARNING

SYLLABUS

Linear Regression Models: Least squares, single & multiple variables, Bayesian linear regression, gradient descent.

Linear Classification Models: Discriminant function – Perceptron algorithm, Probabilistic discriminative model – Logistic regression, Probabilistic generative model – Naive Bayes, Maximum margin classifier – Support vector machine, Decision Tree, Random Forests

SUPERVISED LEARNING

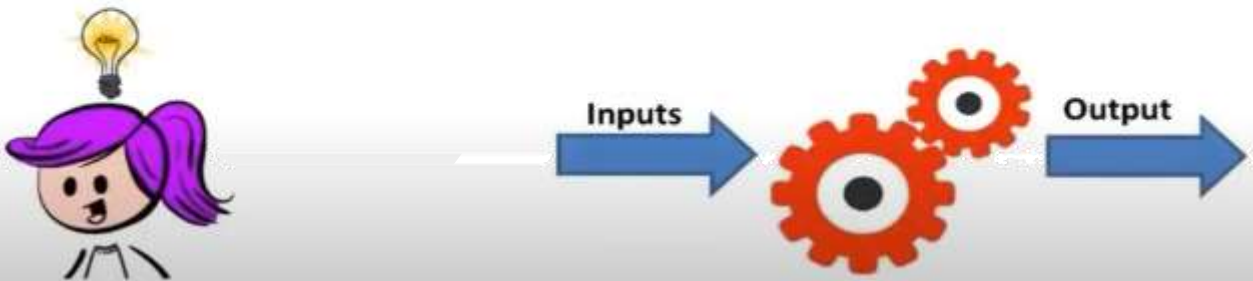
SUPERVISED LEARNING

- It is a paradigm in ML where input objects are mapped to a desired output value train a model.
- The training data is processed, by building a function that maps new data on expected output values.
- It is a type of ML where a model is trained on labeled data.
- Goal → to learn a mapping from inputs to outputs based on the examples provided.
- This approach is called "supervised" because the learning process is guided by the provided labels.
- Input → a vector of predictor variables
- Output value → labels
- MODEL → A mathematical relationship of a real world process in the form of input-output relationship.

4

SUPERVISED LEARNING

Not only models help us **understand** the nature of the **process** being modelled, they also **enable** us to **predict** the **output** based on the **input** features.



5

SUPERVISED LEARNING

Key Concepts in Supervised Learning

- **Labeled Data:** Data that includes input-output pairs, where the output is the label or the target.
- **Training Set:** A subset of the labeled data used to train the model.
- **Test Set:** A subset of the labeled data used to evaluate the model's performance.

6

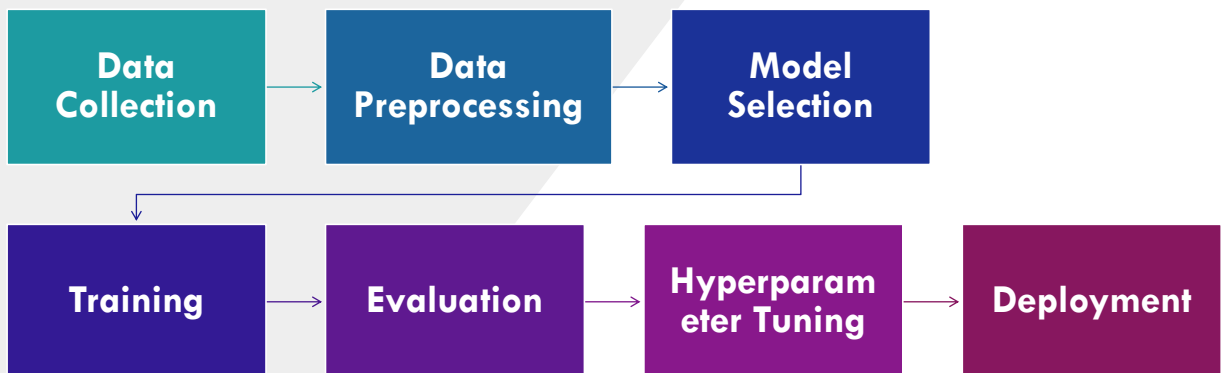
SUPERVISED LEARNING

Steps in Supervised Learning

1. **Data Collection:** Gather a dataset with input-output pairs.
2. **Data Preprocessing:** Clean and prepare the data, which might include normalization, handling missing values, and encoding categorical variables.
3. **Model Selection:** Choose a machine learning algorithm appropriate for the task (e.g., linear regression, decision trees, support vector machines).
4. **Training:** Use the training set to fit the model.
5. **Evaluation:** Test the model using the test set and assess its performance using metrics such as accuracy, precision, recall, F1 score, etc.
6. **Hyperparameter Tuning:** Optimize the model's parameters to improve performance.
7. **Deployment:** Deploy the trained model to make predictions on new, unseen data.

SUPERVISED LEARNING

Steps in Supervised Learning

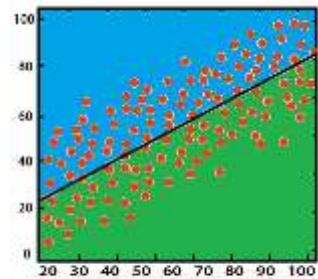


SUPERVISED LEARNING

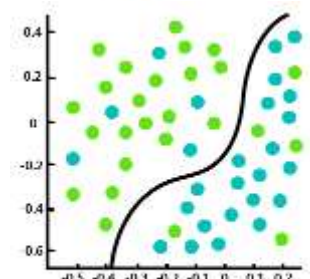
Types of Supervised Learning

- 1. Regression:** The output variable is a continuous value. Examples include predicting house prices, stock prices, etc.
- 2. Classification:** The output variable is a category. Examples include spam detection (spam or not spam), image classification (cat, dog, or bird), etc.

9



Regression



Classification

SUPERVISED LEARNING

Common Algorithms in Supervised Learning

Linear Regression & Polynomial Regression

- Used for regression tasks.

Logistic Regression

- Used for binary regression and classification tasks.

Support Vector Machines (SVM)

- Used for classification tasks.

Decision Trees

- Used for both classification and regression tasks.

k-Nearest Neighbors (k-NN)

- Used for both classification and regression tasks.

Neural Networks

- Used for a wide range of tasks, including image and speech recognition.

Random Forest

- Used for classification

SUPERVISED LEARNING

Regression vs Classification

Regression Algorithm	Classification Algorithm
The output variable has to be a real value or continuous in nature.	The output variable is discrete in the Classification SML problem.
Regression algorithm helps to map the input value and the continuous output variable.	Classification algorithm helps in mapping the input value with the output variable which is discrete in nature.
Regression algorithms are only used for data that is continuous.	Classification algorithms are only used for data that is discrete.
Finds the best fit for accurately predicting the output data in Regression.	Finds the decision boundary which helps in classifying the dataset into different classes in Classification
Regression algorithms help in solving the problems like predicting the weather or the price of Houses in a neighbourhood according to a real estate trend.	Classification algorithms are useful for problems like identifying spam emails, recognition of speech, identifying cancer cells, etc.

LINEAR REGRESSION MODELS:

- **LEAST SQUARES**
- **SINGLE & MULTIPLE VARIABLES**
- **BAYESIAN LINEAR REGRESSION**
- **GRADIENT DESCENT**

REGRESSION

- **What is Regression?**

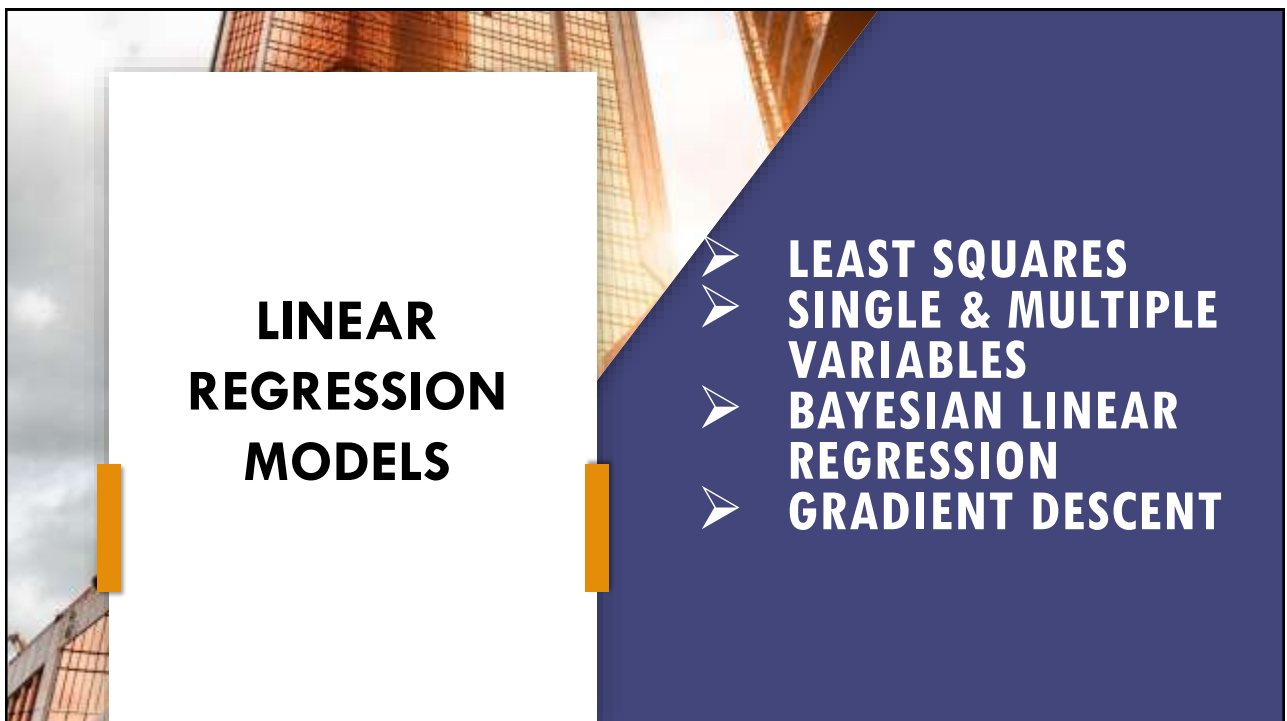
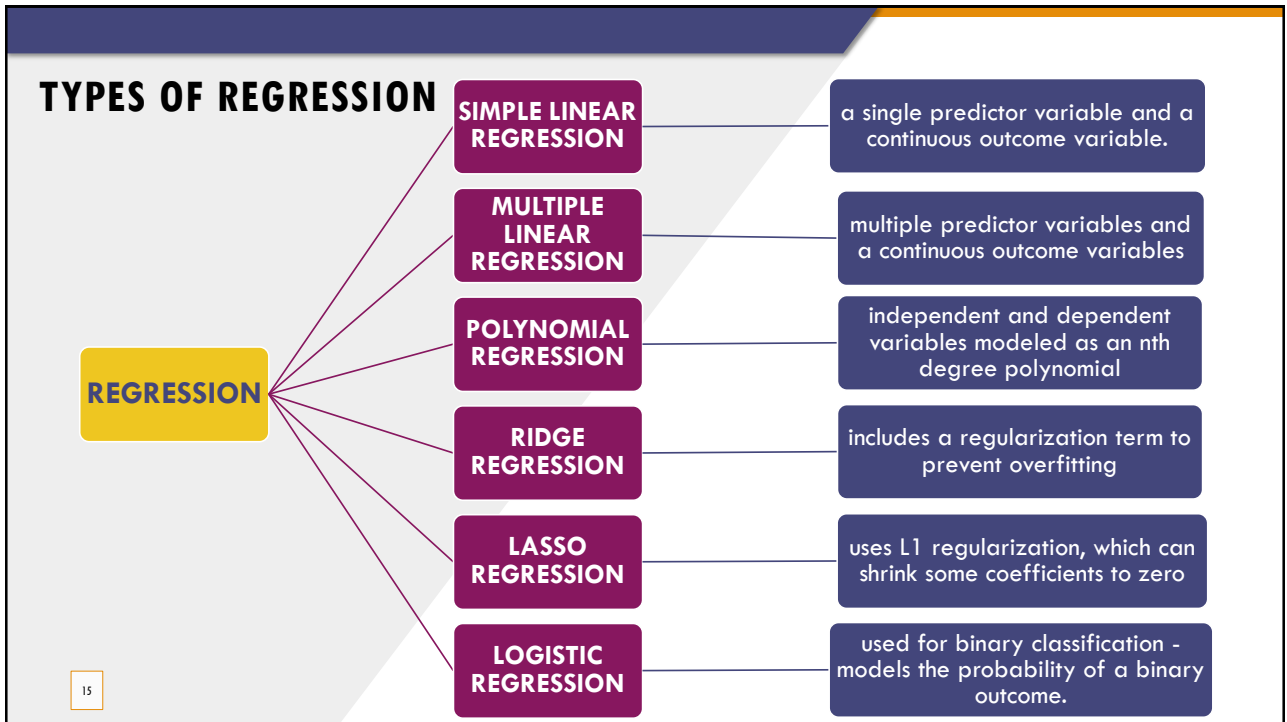
1. Process of finding correlations between dependent and independent variables.
2. A statistical approach used to **analyze the relationship** between a dependent variable (target variable) and one or more independent variables (predictor variables).
3. A type of supervised learning used to **predict** a continuous outcome variable (dependent variable) based on one or more predictor variables (independent variables).
4. Models the relationship between the input features and the target variable → hence helps for the estimation or prediction of numerical values

13

REGRESSION

- **Goal** → to learn a mapping from inputs to outputs that can be used to predict the value of the dependent variable for new data → determine the most suitable function that characterizes the connection between these variables Eg. $y = a_1x_1 + a_2x_2 + b$
- Regression analysis problem works with if output variable is a real or continuous value, such as price, salary or weight.
- Eg. house price prediction, Stock Price Prediction, Risk Management, Disease Progression, Medical Costs, Predictive Diagnostics, Price Optimization, Sales Forecasting, Reliability Engineering, Energy Consumption, Pollution Levels modelling, Water Resource Management, Pest and Disease Forecasting, Behavioral Analysis.

14



LINEAR REGRESSION MODELS

- The simplest form of regression.
- Fundamental tools in machine learning and statistics
- Assumes a linear relationship (or used to model the relationship) between the predictor variables (independent variables) and the outcome variable (dependent or target or response variable).
- Objective → to find a relationship between the input and the target variables
- Performance evaluation measures → Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)

17

LINEAR REGRESSION MODELS

- Simple Linear Regression: $y = \beta_0 + \beta_1 x + \epsilon$
 - Multiple Linear Regression: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$
 - Polynomial Regression: $y = \beta_0 + \beta_1 x^1 + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n + \epsilon$
- $y \rightarrow$ the dependent variable.
- $x_1, x_2, \dots, x_n \rightarrow$ the independent variables.
- $\beta_0 \rightarrow$ the intercept $\rightarrow$ the value of y (response variable) when the values of all the predictor variables, (X_s) are zero
- $\beta_1, \beta_2, \dots, \beta_n \rightarrow$ the coefficients of the independent variables.
- $n \rightarrow$ degree of the polynomial
- $\epsilon \rightarrow$ the error term.

18

LEAST SQUARES

- A standard approach in **regression analysis** to find the **best-fitting curve or line** through a set of points by minimizing the **sum of the squares of the vertical deviations (errors)** between the observed values and the values predicted by the model. $E = \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- Let x be the independent predictor variable and y be the dependent variable.
- Assume that we have a set of observed values of x and y
- A simple linear regression model defines the relationship between x and y using a line defined by an equation in the following form:

$$y = \alpha + \beta x \rightarrow \text{regression line equation}$$

where $y \rightarrow$ target variable, $x \rightarrow$ predictor variable, $\alpha \rightarrow$ y-intercept and $\beta \rightarrow$ slope

- The values of y-intercept and slope are chosen such that they minimize the sum of the squared errors, where

19

$$\text{error} = \text{actual value of } y - \text{predicted value of } y$$

LEAST SQUARES

- Sum of squares of errors (SSE) is

$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n [y_i - (\alpha + \beta x_i)]^2$$

- Where $\hat{y}_i$ is the predicted value of y_i and $y = \alpha + \beta x$
- Now find the values of α and β so that E is minimum
- If $a = \alpha$ and $b = \beta$, E can be calculated using the following eqns.

$$\sum_{i=1}^n y_i = na + b \sum_{i=1}^n x_i$$

- And $\sum_{i=1}^n x_i y_i = a \sum_{i=1}^n x_i + b \sum_{i=1}^n x_i^2$
- Find the values of a and b to calculate E .

20

LEAST SQUARES

How to find the values of a and b and E?

- Find the variance of x $\rightarrow \text{Var}(x)$
- Find the covariance of x and y $\rightarrow \text{Cov}(x, y)$

$$\text{Var}(x) = \frac{1}{n-1} \sum (x_i - \bar{x})^2.$$

$$\text{Cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y}) \quad \bar{x} = \frac{1}{n} \sum x_i \quad \bar{y} = \frac{1}{n} \sum y_i$$

- Now the values of a and b can be calculated as below:

$$b = \frac{\text{Cov}(x, y)}{\text{Var}(x)} \quad a = \bar{y} - b\bar{x}$$

- The final regression equation $\rightarrow y = a + bx$
- Now calculate $\hat{y}_i$ for each y_i
- And compute SSE as $E = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ and $\text{MSE} = \text{SSE}/n$ and $\text{RMSE} = \sqrt{\text{MSE}}$

21

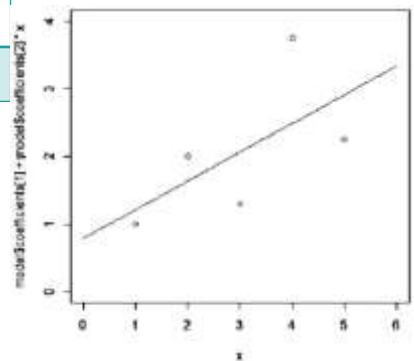
LEAST SQUARES

- Example:**
- Obtain a linear regression for the data in the following table assuming that y is the independent variable.

X	1.0	2.0	3.0	4.0	5.0
Y	1.00	2.00	1.30	3.75	2.25

- Solution: the linear regression model for the given data

$$y = 0.785 + 0.425x$$



22

LEAST SQUARES

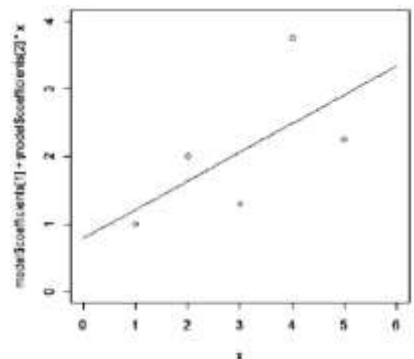
Steps:

- Calculate the variance(x)
- Calculate the covariance(x, y)
- Calculate the value of b.
- Using b, calculate the value of a.
- Substitute the values of a and b in $y = ax + b$
- Get the predicted values of y as $\hat{y}$.
- To evaluate the model performance, find the SSE, MSE and RMSE.

23

LEAST SQUARES

X_i	Y_i	$(X_i - \bar{X})$	$(Y_i - \bar{Y})$	$(X_i - \bar{X})^2$	$(X_i - \bar{X})(Y_i - \bar{Y})$
1.0	1.00	-2	-1.06	4	2.12
2.0	2.00	-1	-0.06	1	0.06
3.0	1.30	0	-0.76	0	0
4.0	3.75	1	1.69	1	1.69
5.0	2.25	2	0.19	4	0.38
$\bar{X} = 3.0$	$\bar{Y} = 2.06$			10	4.25



$$\text{Cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y}) = 1.0625$$

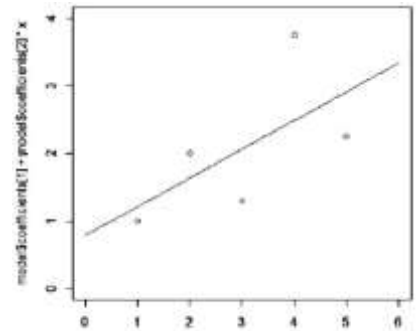
$$\text{Var}(x) = \frac{1}{n-1} \sum (x_i - \bar{x})^2 = 2.5$$

$$b = 1.0625 / 2.5 = 0.425 \quad a = 2.06 - (0.425)(3.0) = 0.785$$

- The linear regression model for the given data: $y = 0.785 + 0.425x$

LEAST SQUARES

X_i	Y_i	$\hat{Y}_i$	$(Y_i - \hat{Y}_i)$	$(Y_i - \hat{Y}_i)^2$
1.0	1.00	1.21	-0.21	0.0441
2.0	2.00	1.635	0.37	0.133225
3.0	1.30	2.06	-0.76	0.5776
4.0	3.75	2.485	1.27	1.600225
5.0	2.25	2.91	-0.66	0.4356
				2.791



$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Substitute x_i in $y = 0.785 + 0.425x$ to compute $\hat{Y}_i$
- Then calculate SSE, Sum of squares of errors = 2.791
- Mean square error = MSE = SSE / n = 0.558
- Root mean square error = RMSE = $\sqrt{\text{MSE}}$ = 0.747

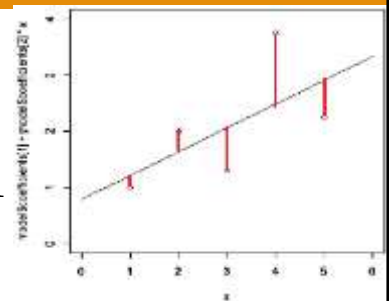
LEAST SQUARES

- Sum of squares of errors = $E = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = 2.791$

- The red lines in the graph denote the deviation of each data point from the predicted or estimated value (regression line)
- Error is the deviation of the data points from the imaginary line (regression line), $y = a + bx$
- Estimation $\rightarrow$ The process of getting parameter estimators (a and b)
- Least squares method $\rightarrow$ estimation method of Ordinary Least Squares (OLS)

Disadvantages:

- Lack robustness to outliers
- All datasets not suitable for least squares method



LEAST SQUARES

Variations:

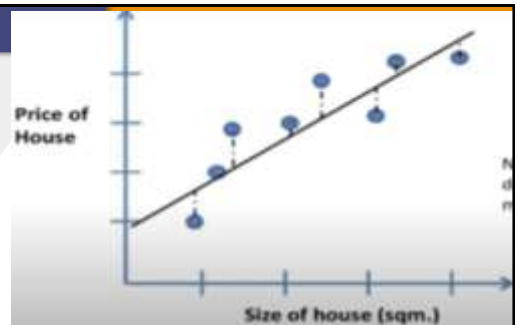
- Weighted least squares method → the coefficients a and b are estimated such that the weighted sum of squares of errors is minimum for some positive constants $w_1, \dots, w_n$.

$$E = \sum_{i=1}^n w_i [y_i - (a + bx_i)]^2$$

- Generalised least squares method
- Partial least squares method
- Total least squares method

27

SINGLE VARIABLE REGRESSION



- Assume that the number of independent variables is 1 → x
- Let the dependent variable be y .
- Here fit a line to data, from classification problems, where we find a line that separates out the classes, so that they can be distinguished.
- The simple linear regression model using single variable defines the relationship between the independent variable x and the dependent variable by an equation of the following form:

$$y = \beta_0 + \beta_1 x + \epsilon$$

28

MULTIPLE VARIABLES REGRESSION

- Regression analysis → used to predict the value of one or more responses from a set of predictors.
- Also used to estimate the linear association between the predictors and responses → predictors can be continuous or categorical or a combination of both.
- Multiple independent variables affect the response variable → called as multiple linear regression
- Assume that there are N independent variables $x_1, x_2, \dots, x_N$. Let the dependent variable be y .
- Let there also be n observed values of these variables:
- The multiple linear regression model defines the relationship between the N independent variables and the dependent variable by an equation of the following form: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$

29

MULTIPLE VARIABLES REGRESSION

- In many applications, several measurements are made on each individual or event generating an observation vector.
- The sample may be viewed as a data matrix as below
- Here the d columns correspond to d variables denoting the result of measurements made on an individual or event → also called inputs, features, or attributes.
- The N rows correspond to independent and identically distributed observations, examples, or instances on N individuals or events.

$$\mathbf{X} = \begin{bmatrix} X_1^1 & X_2^1 & \dots & X_d^1 \\ X_1^2 & X_2^2 & \dots & X_d^2 \\ \vdots & \vdots & \ddots & \vdots \\ X_1^N & X_2^N & \dots & X_d^N \end{bmatrix}$$

30

MULTIPLE VARIABLES REGRESSION

- Use the ordinary least squares method to obtain the optimal estimates of $\beta_0, \beta_1, \beta_2, \dots, \beta_n$
- Procedure for the computation of these optimal estimates. Let

$$X = \begin{bmatrix} 1 & x_{11} & x_{21} & \cdots & x_{N1} \\ 1 & x_{12} & x_{22} & \cdots & x_{N2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{1n} & x_{2n} & \cdots & x_{Nn} \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad B = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_N \end{bmatrix}$$

- Then the regression coefficients are given by

$$B = (X^T X)^{-1} X^T Y$$

31

MULTIPLE VARIABLES REGRESSION

Example:

- Fit a multiple linear regression model to the following data:

X1	1.0	1.0	2.0	0.0
X2	1	2	2	1
Y	3.25	6.5	3.5	5.0

- Number of independent variables = $n = 2$
- Number of observations = $N = 4$
- Hence multiple linear regression model for this problem has the form $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$

32

MULTIPLE VARIABLES REGRESSION

Solution:

- The computation: Transform the given dataset as below

$$\bullet \quad X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 0 & 1 \end{bmatrix} \quad Y = \begin{bmatrix} 3.25 \\ 6.5 \\ 3.5 \\ 5.0 \end{bmatrix} \quad B = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

- Find the coefficients using $B = (X^T X)^{-1} X^T Y$

- Answer: The required model is

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$y = 2.0625 - 2.3750x_1 + 3.2500x_2$$

33

MULTIPLE VARIABLES REGRESSION

Solution:

$$\bullet \quad X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 0 & 1 \end{bmatrix} \quad X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 0 \\ 1 & 2 & 2 & 1 \end{bmatrix} \quad X^T X = \begin{bmatrix} 4 & 4 & 6 \\ 4 & 6 & 7 \\ 6 & 7 & 10 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{|X^T X|} \text{Adj}(X^T X)$$

$$\bullet \quad |X^T X| = 4(60 - 49) - 4(40 - 42) + 6(28 - 36) = 4(11) - 4(-2) + 6(-8) = 4$$

$$\bullet \quad \text{Adj}(X^T X) = \begin{bmatrix} (60 - 49) & -(40 - 42) & (28 - 36) \\ -(40 - 42) & (40 - 36) & -(28 - 24) \\ (28 - 36) & -(28 - 24) & (24 - 16) \end{bmatrix} = \begin{bmatrix} 11 & 2 & -8 \\ 2 & 4 & -4 \\ -8 & -4 & 8 \end{bmatrix}$$

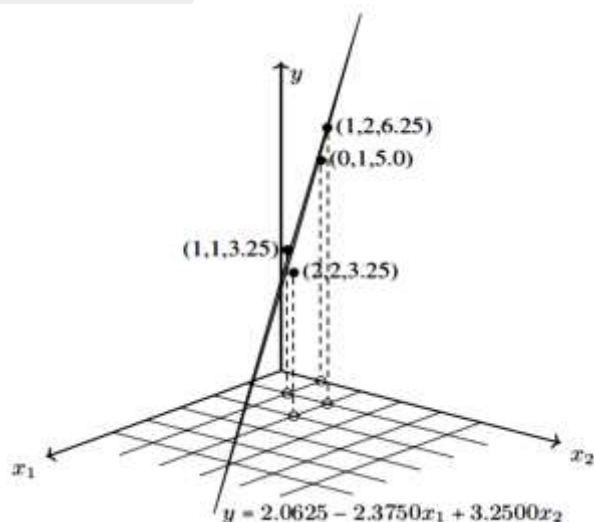
34

MULTIPLE VARIABLES REGRESSION

- $(X^T X)^{-1} = \frac{1}{|X^T X|} \text{Adj}(X^T X) = \frac{1}{4} \begin{bmatrix} 11 & 2 & -8 \\ 2 & 4 & -4 \\ -8 & -4 & 8 \end{bmatrix} = \begin{bmatrix} 11/4 & 1/2 & -2 \\ 1/2 & 1 & -1 \\ -2 & -1 & 2 \end{bmatrix}$
- $(X^T X)^{-1} X^T = \begin{bmatrix} 11/4 & 1/2 & -2 \\ 1/2 & 1 & -1 \\ -2 & -1 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 0 \\ 1 & 2 & 2 & 1 \end{bmatrix}$
 $= \begin{bmatrix} 5/4 & -3/4 & -1/4 & 3/4 \\ 1/2 & -1/2 & 1/2 & -1/2 \\ -1 & 1 & 0 & 0 \end{bmatrix}$
- $(X^T X)^{-1} X^T Y = \begin{bmatrix} 5/4 & -3/4 & -1/4 & 3/4 \\ 1/2 & -1/2 & 1/2 & -1/2 \\ -1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 3.25 \\ 6.5 \\ 3.5 \\ 5.0 \end{bmatrix} = \begin{bmatrix} 2.0625 \\ -2.375 \\ 3.25 \end{bmatrix} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = B$
- The required model is $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \rightarrow y = 2.0625 - 2.375x_1 + 3.25x_2$

MULTIPLE VARIABLES REGRESSION

- The Regression plane for the data:



MULTIPLE VARIABLES REGRESSION

SIMPLE LINEAR REGRESSION	MULTIPLE LINEAR REGRESSION
One dependent variable y predicted from one independent variable x .	One dependent variable y predicted from set of independent variables $x_1, x_2, \dots, x_n$
Regression line equation: $y = \beta_0 + \beta_1 x + \epsilon$	Regression line equation: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$
One regression coefficient	One regression coefficient for each variable

37

BAYESIAN LINEAR REGRESSION

- **Bayesian Linear Regression** is a probabilistic approach to linear regression that **treats the model parameters as random variables** rather than fixed values.
- To deal with insufficient data or poorly distributed data.
- Produces a **distribution over possible values** for the weights, giving a way to **quantify uncertainty** in predictions
- In the Bayesian viewpoint, linear regression is formulated using probability distributions rather than point estimates.
- The response, y , is not estimated as a single value, but is assumed to be drawn from a probability distribution.
- The aim of Bayesian Linear Regression → not to find the single “best” value of the model parameters, but rather to determine the posterior distribution for the model parameters.

38

BAYESIAN LINEAR REGRESSION

- The result, "y," is produced by a normal distribution ($N(0, \sigma^2)$ where the variance and mean are normalized).
- The goal of the Bayesian Regression Model is to identify the 'posterior' distribution again for model parameters rather than the model parameters themselves.
- The model parameters will be expected to follow a distribution in addition to the output y.

39

BAYESIAN LINEAR REGRESSION

- The posterior expression: **Posterior = (Likelihood * Prior)/Normalization**
- The expression parameters:
 - Posterior: It is the likelihood (chance) that an event, such as H, will take place given the occurrence of another event, such as E, i.e., $P(H | E)$.
 - Likelihood: It is a likelihood function in which a marginalization parameter variable is used.
 - Prior: This refers to the likelihood that event H happened before event A, i.e., $P(H)$ (H)
- This is the same as Bayes' Theorem, which states the following -

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- Here, A and B seem to be events.
- $P(A)$ is the likelihood that event A will occur;
- $P(B)$, the likelihood of event B happening (cannot be zero because it already has happened)
- $P(B|A)$ is the likelihood that event B occurred, provided that event A has already occurred.
- $P(A|B)$ is the likelihood that event A will occur, provided that event B has already occurred.

BAYESIAN LINEAR REGRESSION

1. Prior Distribution

- Represents our initial beliefs about the model parameters (weights).
- Typically chosen as a Gaussian distribution.

2. Likelihood

- Based on the observed data and assumes a Gaussian distribution of errors.
- Describes how likely the observed data is for a given set of weights.

3. Posterior Distribution

- The updated belief about the parameters after seeing the data.
- Calculated using Bayes' Theorem:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

4. Predictive Distribution

- Instead of predicting a single value, Bayesian regression gives a distribution of predicted values, allowing for **uncertainty estimation**.

EG. To predict house prices using square footage

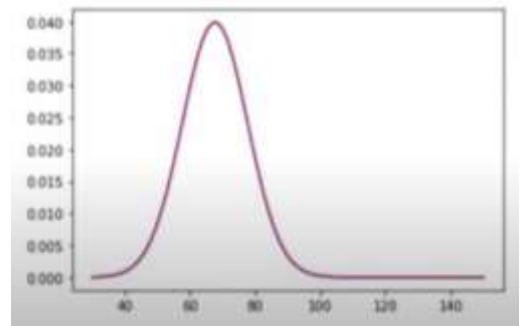
- **Prior:** We believe most weights are small, centered around 0 (Gaussian prior).
- **Likelihood:** Based on training data (sqft, price), assuming Gaussian noise.
- **Posterior:** Updated distribution over weights after seeing the data.
- **Prediction:** For a new house, we get a **mean predicted price** and a **confidence interval**.

BAYESIAN LINEAR REGRESSION

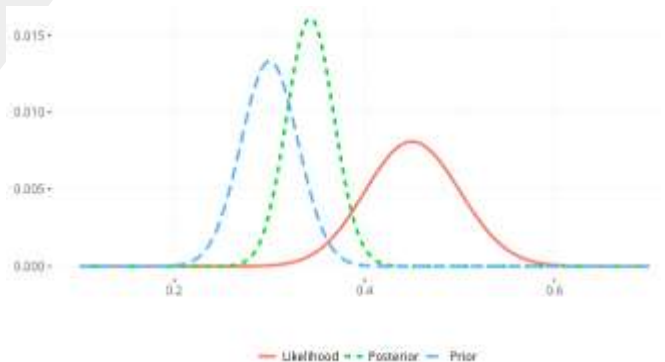
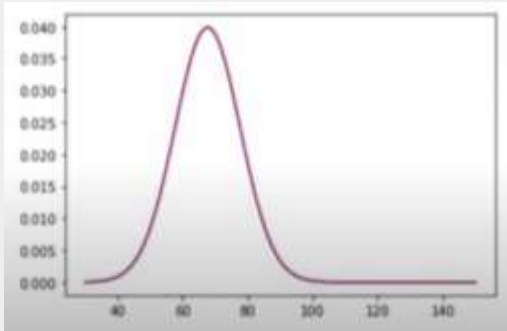
- The response, y , is not estimated as a single value, but is assumed to be drawn from a probability distribution.

$$y \sim N(\beta^T X, \sigma^2 I)$$

- $N \rightarrow$ Normal distribution $\rightarrow N(\text{mean}, \text{variance})$
- $\beta \rightarrow$ weight / coefficient / slope vector
- $X \rightarrow$ input vector
- $\sigma^2 \rightarrow$ variance
- $I \rightarrow$ Identity matrix



BAYESIAN LINEAR REGRESSION



43

BAYESIAN LINEAR REGRESSION

- The response (y) generated from a probability distribution, but the model parameters (β) are assumed to come from a distribution as well.
- The posterior probability of the model parameters (β) is **conditional** upon the training inputs and outputs:

$$P(\beta|y, X) = \frac{P(y|\beta, X) * P(\beta|X)}{P(y|X)}$$

- $P(\beta|y, X)$ is the posterior probability distribution of the model parameters given the inputs and outputs.
- This is equal to the likelihood of the data, $P(y|\beta, X)$, multiplied by the prior probability of the parameters and divided by a normalization constant.

44

BAYESIAN LINEAR REGRESSION

Benefits of Bayesian Linear Regression.

- **Priors:** If we have domain knowledge, or a guess for what the model parameters should be, we can include them in our model, unlike in the frequentist approach which assumes everything there is to know about the parameters comes from the data. If we don't have any estimates ahead of time, we can use non-informative priors for the parameters such as a normal distribution.
- **Posterior:** The result of performing Bayesian Linear Regression is a distribution of possible model parameters based on the **data** and the **prior**. This allows us to quantify our uncertainty about the model: if we have fewer data points, the posterior distribution will be more spread out.
- Bayesian regression can be very useful when we have **insufficient data** in the dataset or the data is **poorly distributed**.
- As the amount of data points increases, the likelihood washes out the prior, and in the case of infinite data, the outputs for the parameters converge to the values obtained from OLS.
- The formulation of model parameters as distributions encapsulates the Bayesian worldview: we start out with an initial estimate, our prior, and as we gather more evidence, our model becomes less wrong.

BAYESIAN LINEAR REGRESSION

Disadvantages Of Bayesian Regression

- The model's inference process can take some time.
- The Bayesian strategy is not worthwhile if there is a lot of data accessible for our dataset, and the regular probability approach does the task more effectively.

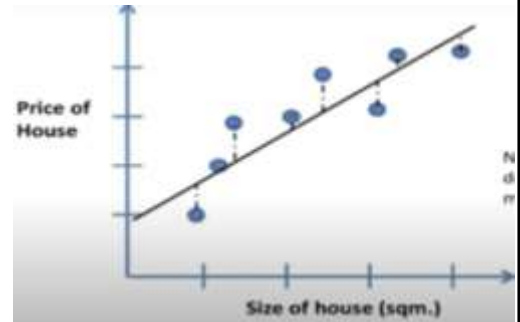
Real-life Application Of Bayesian Linear Regression

- **Using Priors:** Consider a scenario in which your supermarkets carry a new product, and we want to predict its initial Christmas sales. For the new product's Christmas effect, we may merely use the average of comparable things as a previous one → once we obtain data from the new item's initial Christmas sales, the previous is immediately updated. As a result, the forecast for the next Christmas is influenced by both the prior and the new item's data.
- **Regularize Priors:** With the season, day of the week, trend, holidays, and a tonne of promotion indicators, our model is severely over-parameterized. Therefore regularization is crucial to keep the forecasts in check.

GRADIENT DESCENT

- Sum of squares of errors (SSE):
- y_i - actual value, $\hat{y}_i$ - predicted value
- Mean Squared Error (MSE) = $\frac{SSE}{n}$
- Cost function = measure of distance of our predicted model (regression line) from the data points = $\frac{SSE}{2n}$
- Lesser is the distance $\rightarrow$ better is the model
- The aim of linear regression $\rightarrow$ to minimize cost function $\rightarrow$ to fit the best regression line with reduced MSE
- How to minimize cost function?

$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n [y_i - (\alpha + \beta x_i)]^2$$



GRADIENT DESCENT

$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n [y_i - (\alpha + \beta x_i)]^2$$

- All lines ($y = \alpha + \beta x$) are just different combinations of α (intercept) and β (slope) $\rightarrow$ hence find optimized values for α and β that will minimize MSE and the cost function
- **Cost function** $\rightarrow$ defined as the measurement of difference or error between actual values and expected values at the current position and present in the form of a single real number.
- By changing α and β the cost function can be changed
- Keep changing (iterate) till a combination of α and β that minimizes the cost function is obtained.
- Gradient descent algorithm used for this...

GRADIENT DESCENT

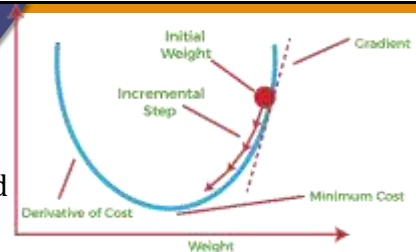
- **Gradient Descent** → an optimization algorithm used to minimize a function by iteratively moving in the direction of the steepest descent (i.e., the negative gradient) → to train ML and DL models by means of minimizing errors between actual and expected results → used to minimize the loss function and update model parameters (weights).
- Also used to train Neural Networks.
- If $y = f(x)$ is the objective function, gradient descent alg. tries to minimize y (or) the cost function parameterized by the model's parameters.
- Find $\frac{\Delta y}{\Delta x}$ or $\frac{\partial y}{\partial x}$ → change in y w.r.t. x → partial derivative of y wrt x
- Helps in finding the local minimum of a function.
- The main objective of using a gradient descent algorithm → to minimize the cost function using iteration.

49

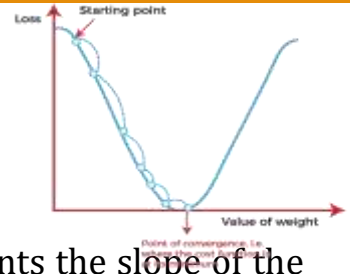
GRADIENT DESCENT

- Gradient → measures the change in all weights with regard to the change in error.
- If a graph is plotted between error and slope / coefficient, it gives a convex curve.
- Let us consider one point a_1
- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the local minimum of that function.
- If we move towards a positive gradient or towards the gradient of the function at the current point, it will give the local maximum of that function.
- Iterate the following steps:
 - Calculate the first-order derivative of the function to compute the gradient or slope of that function.
 - Move away from the direction of the gradient → slope increased from the current point by α times (where α is defined as Learning Rate → a tuning parameter in the optimization process which helps to decide the length of the steps).

50



GRADIENT DESCENT



- **Procedure:**
- In a Simple linear regression: $y = mx + c$, 'm' represents the slope of the line, and 'c' represents the intercepts on the y-axis.
- Consider an arbitrary point on the curve.
- Derive the first derivative or slope - then use a tangent line to calculate the steepness of this slope.
- This slope will update the parameters (weights and bias), initially wrt the first arbitrary point and then then subsequent points.
- As we descend towards the higher weights, the steep slope approaches the lowest point, which is called a point of convergence (slope is 0).
- Learning continued till slope becomes 0.

GRADIENT DESCENT



- Let the arbitrary point be a_1 , representing the current position.
- If b is the next position, then $b = a_1 - \lambda f(a_1)$, where $\lambda \rightarrow$ a small fixed quantity in the range of 0.01 (If λ is very large, it will fail to converge).
- Hence size of step (or) value of b depends on $f(a)$.
- If slope is higher, the steps are larger and hence b is far from a_1 .
- As the step gets repeated, slope decreases, and we move towards the minima and hence step size decreases.
- At minimum, slope become 0.
- Eg. a blindfolded man walking downwards in a hill. His step size reduces as walks in a steeper (slope decreased) region.
- Multiple iterations depending on Y and starting point a_1
- In the same way, GD alg. tries to minimize cost function by changing α and β

GRADIENT DESCENT

$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n [y_i - (\alpha + \beta x_i)]^2$$

- Let $c = \alpha$ and $m = \beta_i$
- $SSE = \frac{1}{n} \sum_{i=1}^n (y_i - (\alpha + \beta x_i))^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + c))^2$
- Let $J = SSE$, hence J is a function of $(m, c) \rightarrow J(m, c)$
- Differentiating J wrt m
- $\frac{\partial J}{\partial m} = \frac{2}{n} \sum_{i=1}^n (-x_i)(y_i - mx_i - c)$
- This must be equated to 0 $\rightarrow \frac{\partial J}{\partial m} = 0$
- $\frac{2}{n} \sum_{i=1}^n (-x_i)(y_i - mx_i - c) = 0$
- $\sum_{i=1}^n (-x_i y_i + mx_i^2 + x_i c) = 0$
- $\sum_{i=1}^n mx_i^2 + \sum_{i=1}^n x_i c = \sum_{i=1}^n (x_i y_i)$

$$J = \frac{1}{n} \sum_{i=1}^n (y_i - mx_i - c)^2$$

$$m \sum_{i=1}^n x_i^2 + c \sum_{i=1}^n x_i = \sum_{i=1}^n (x_i y_i)$$

53

GRADIENT DESCENT

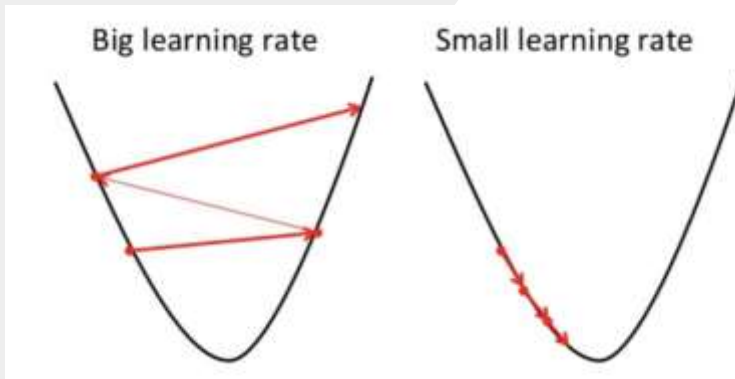
$$J = \frac{1}{n} \sum_{i=1}^n (y_i - mx_i - c)^2$$

- Differentiating J wrt c
- $\frac{\partial J}{\partial c} = \frac{2}{n} \sum_{i=1}^n -(y_i - mx_i - c)$
- This must be equated to 0 $\rightarrow \frac{\partial J}{\partial c} = 0$
- $\frac{2}{n} \sum_{i=1}^n (-y_i + mx_i + c) = 0$
- $m \sum_{i=1}^n x_i + \sum_{i=1}^n c = \sum_{i=1}^n (y_i)$
- Now the ML algorithm should know that with a finite number of steps, it can converge to a minimum $\rightarrow$ learnt using parameter 'learning rate'.
- Incorporate these equations to find the updated values of m and c to find the learning rate (the number of iterations taken to reach convergence point) – using an infinitely small parameter value λ .
- After each iteration, the m and c values are updated as below:
- $m = m - \lambda \frac{\partial J}{\partial m}$ and $c = c - \lambda \frac{\partial J}{\partial c}$

54

GRADIENT DESCENT

- The ML algorithm should know that with a finite number of steps, it can converge to a minimum $\rightarrow$ learnt using an infinitely small parameter value $\rightarrow \lambda \rightarrow$ 'learning rate'.



55

GRADIENT DESCENT

Eqn. 1 $\rightarrow m \sum_{i=1}^n x_i^2 + c \sum_{i=1}^n x_i = \sum_{i=1}^n (x_i y_i)$

Eqn. 2 $\rightarrow m \sum_{i=1}^n x_i + \sum_{i=1}^n c = \sum_{i=1}^n (y_i)$

- Given the 2 equations, let us estimate the values for m and c that fit for the following 5 observations:

x_i	y_i
1	1
3	3
5	4
7	3
9	5

56

GRADIENT DESCENT

Eqn. 1 → $m \sum_{i=1}^n x_i^2 + c \sum_{i=1}^n x_i = \sum_{i=1}^n (x_i y_i)$

- $n = 5$.
- Find the $x_i y_i$ and x_i^2 and summation values.
- Substitute the values in eqn.1:
 $m(165) + c(25) = 96 \leftarrow \text{Eqn. 3}$
- Now substitute the values in eqn.2:
 $m(25) + 5c = 16 \leftarrow \text{Eqn. 4}$
- Solving eqns. 3 and 4:
- $m = 2/5$ and $c = 6/5$
- Hence $y = \frac{2}{5}x + \frac{6}{5}$

Eqn. 2 → $m \sum_{i=1}^n x_i + \sum_{i=1}^n c = \sum_{i=1}^n (y_i)$

	x_i	y_i	$x_i y_i$	x_i^2
1.	1	1	1	1
2.	3	3	9	9
3.	5	4	20	25
4.	7	3	21	49
5.	9	5	45	81
Σ	25	16	96	165

GRADIENT DESCENT

Types of Gradient Descent

1. Batch Gradient Descent (BGD)

- Also called vanilla gradient descent - used to find the error for each point in the training set and update the model after evaluating all training examples → process is like a cycle and called a training epoch. **Advantage:** good computational efficiency **Disadvantage:** unsuitable state of convergence sometimes, training set must be in local memory

2. Stochastic Gradient Descent (SGD)

- Calculates the error for each training sample within the dataset → it updates the parameters for each training example one by one. faster than BGD. **Advantage:** detailed rate of improvement. **Disadvantage:** computationally expensive, noisy gradients possible

3. Mini-Batch Gradient Descent

- Combination of the concepts of SGD and BGD - splits the training dataset into small batches and performs an update for each of those batches - used with deep learning.

LINEAR CLASSIFICATION MODELS:

- **DISCRIMINANT FUNCTION**
- **PERCEPTRON ALGORITHM**
- **PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION**
- **PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES**
- **MAXIMUM MARGIN CLASSIFIER – SUPPORT VECTOR MACHINE, DECISION TREE, RANDOM FORESTS**

LINEAR CLASSIFICATION MODELS

- Linear classification models → a type of ML model used for classifying data points into categories.
- In machine learning, classification is the problem of identifying to which of a set of categories a new observation belongs, on the basis of a training set of data containing observations (or instances) whose category membership is known.
- The key characteristic of these models is that they make their predictions based on a linear combination of input features.
- A classification algorithm makes its classification based on linear prediction function combining a set of weights with the feature vector.
- Makes the classification decision based on the value of a linear combination of the characteristic features.

LINEAR CLASSIFICATION MODELS

How They Work:

- **Decision Boundary:** Linear classification models create a decision boundary that separates classes. This boundary is a hyperplane (a line in 2D, a plane in 3D, etc.).
- **Prediction:** A new data point is classified based on which side of the decision boundary it falls on.

Examples of Linear Classification Models:

- Logistic Regression
- Support Vector Machine (SVM)
- Perceptron
- Linear Discriminant Analysis (LDA)

Applications: Spam Detection, Medical Diagnosis, Sentiment Analysis, Optical character recognition, Face / speech recognition

LINEAR CLASSIFICATION MODELS

Example:

- Consider the following data:
- 2 attributes → “Score1” and “Score2”.
- Class label → “Result” - has two possible values “Pass” and “Fail”.
- The data can be divided into two categories or classes: The set of data for which the class label is “Pass” and the set of data for which the class label is “Fail”.
- Which of the two categories or classes the new observation should be assigned?
- To answer this question, using the given data alone we need to find the rule, or the formula, or the method that has been used in assigning the values to the class label “Result”.
- The problem of finding this rule or formula or the method is the classification problem.

Score1	29	22	10	31	17	33	32	20
Score2	43	29	47	55	18	54	40	41
Result	Pass	Fail	Fail	Pass	Fail	Pass	Pass	Pass

DISCRIMINANT FUNCTION

Score1	29	22	10	31	17	33	32	20
Score2	43	29	47	55	18	54	40	41
Result	Pass	Fail	Fail	Pass	Fail	Pass	Pass	Pass

- Discriminant of a classification problem → a rule or a function that is used to assign labels to new observations.
- A mathematical function used in statistical classification models to separate different classes or groups in a dataset.
- It assigns a score to each data point, which is used to decide the class to which the data point belongs.
- The primary goal of a discriminant function is to find the boundary that best separates different classes in a multidimensional feature space.
- Example: consider the following rules for the classification of the new data for the above example dataset:

IF Score1 + Score2 ≥ 60, THEN "Pass" ELSE "Fail".

IF Score1 ≥ 20 AND Score2 ≥ 40 THEN "Pass" ELSE "Fail".

- These rules can be generalized for some M, m1, m2 as below:

IF Score1 + Score2 ≥ M, THEN "Pass" ELSE "Fail".

IF Score1 ≥ m1 AND Score2 ≥ m2 THEN "Pass" ELSE "Fail".

63

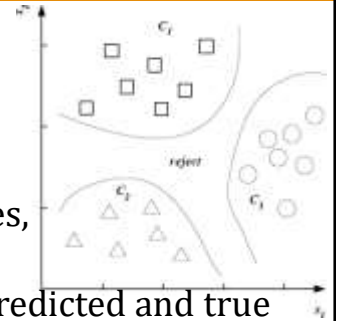
DISCRIMINANT FUNCTION

- 2 types → Linear Discriminant function or Quadratic Discriminant Function
- In **Linear Discriminant Analysis (LDA)**, the discriminant function is linear → it is a linear combination of the input features.
- The function takes the form:

$$y(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- where $x_1, x_2, \dots, x_n$ are the features, and $w_0, w_1, \dots, w_n$ are the weights or coefficients.
- The decision boundary between two classes is determined by the condition $y(x)=0$.
- Learning a discriminant function → finding the weights w that best separates the classes.

DISCRIMINANT FUNCTION



- “Best” → means finding the weights that
 - minimize the total number of misclassified examples,
 - maximize class separability,
 - minimize the sum of squared errors between the predicted and true responses, or
 - producing a decision boundary that has maximum margin.
- Objective function → cost or loss function → quantifies how good/bad a set of parameters are compared with others
- Objective function can then be used to train our model and find the optimal parameters.
- Once a model is fitted, it can be used to make predictions on new test data.

DISCRIMINANT FUNCTION

- LDA → one of the commonly used dimensionality reduction techniques in ML to solve multi-class classification problems.
- Works by identifying a linear combination of features that separates or characterizes two or more classes of objects or events.
- Projects data with two or more dimensions into one dimension so that it can be more easily classified → technique referred to as **dimensionality reduction**.
- LDA algorithms make predictions by using Bayes to calculate the probability of whether an input data set will belong to a particular output
- When the number of classes, $K = 2$, the classification system is a dichotomizer and for $K \geq 3$, it is a polychotomizer.

DISCRIMINANT FUNCTION

A practical application of LDA

- A bank deciding whether to approve or reject loan applications.
- Uses two features to make this decision → the applicant's credit score and annual income.
- The two features or classes are plotted on a 2-dimensional (2D) plane with an X-Y axis.
- By applying LDA, a straight line that completely separates these two class data points is drawn.
- LDA achieves this by using the X-Y axis to create a new axis, separating the different classes with a straight line and projecting data onto the new axis.
- To create this new axis and reduce dimensionality, LDA follows these criteria:
 - Maximize the distance between the means of two classes.
 - Minimize the variance within individual classes.

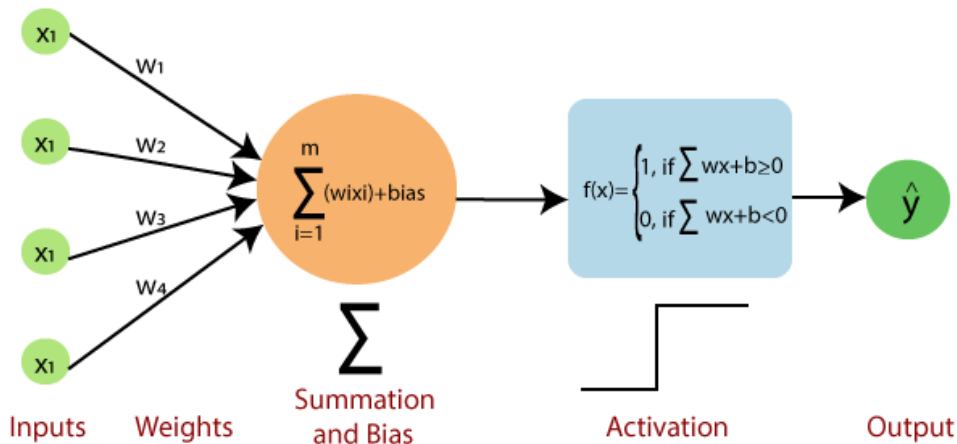
67

PERCEPTRON ALGORITHM

- The Perceptron algorithm is one of the earliest and simplest types of artificial neural networks (ANN).
- It is a binary classifier → a supervised learning model that categorizes data into one of two classes or categories.
- The perceptron takes several input features, multiplies each by a weight, sums them up, adds a bias, and then applies an activation function (usually the step function) to decide the output (0 or 1).
- It aims to find the optimal weights and bias so that the classifier correctly predicts the output for training data.
- Works only for linearly separable data (data that can be divided by a straight line or hyperplane).
- Simple, interpretable, and fast to train.
- Basis for more advanced models like multilayer perceptrons.

68

PERCEPTRON ALGORITHM



69

PERCEPTRON ALGORITHM

How a perceptron algorithm works?

1. Initialize with random small weights and bias.
2. For each training example
 - i. Compute weighted sum: $z = \sum_{i=1}^n w_i x_i + b$
 - ii. Apply activation function: $f(x)$
 - iii. Apply step function: $\hat{y} = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ 0 & \text{Otherwise} \end{cases}$
 - iv. Update weights and bias if prediction is wrong:
 - a) $w_i = w_i + \lambda (y - \hat{y}) x_i$
 - b) $b = b + \lambda (y - \hat{y})$
3. Repeat Step-2 until no errors (or maximum iterations reached)

70

PROBABILISTIC MODELS

- Probabilistic models in machine learning → statistical frameworks that explicitly incorporate uncertainty and randomness into their predictions and analyses.
- Unlike deterministic models that provide a single, definitive output for a given input, probabilistic models quantify the likelihood of different outcomes.
- Characteristics:
 - Handling Uncertainty
 - Probability Distributions
 - Inference
 - Quantifying Certainty
- Common models: Logistic regression, Bayesian Networks, Hidden Markov Models, Gaussian Processes, and various forms of probabilistic deep learning models.

71

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

- Two fundamentally different approaches in machine learning for modeling the relationship between input features X and output labels Y :

1. Discriminative Models

- Types of ML tools used for tasks like predicting or classifying things.
- Work by guessing what might happen based on the information they're given → Directly model $P(Y|X)$ → the probability of a label given the input
- They just make predictions about the ones they already know about.
- Algorithms: Logistic Regression, Support Vector Machines (SVM), Decision Tree, Random Forest, Neural Network

2. Generative Models

- Learn the overall pattern and can generate new data similar to the training data → model the joint probability: $P(X,Y)=P(X|Y)P(Y)$
- Useful for things like making new images, writing text, or even beefing up a dataset with more examples
- Algorithms: Naive Bayes, Hidden Markov Model (HMM), Gaussian Mixture Models (GMM), Variational Auto Encoder (VAE), Generative Adversarial Networks (GANs)

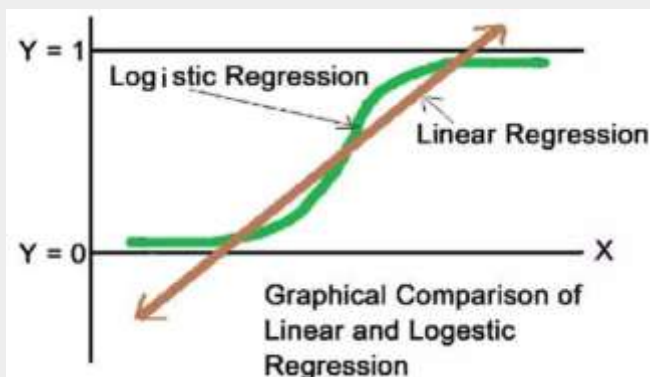
72

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

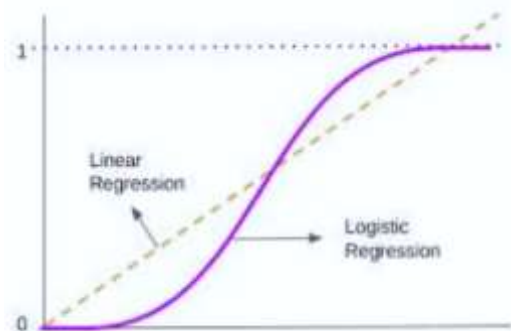
- A type of discriminative model (conditional model) that uses a probabilistic framework to model the relationship between predictor variables and a binary response variable - means it's useful for supervised learning tasks
- It can account for uncertainty in the data and model parameters.
- LR is often used for binary classification problems, such as assigning labels like "pass/fail", "win/lose", "spam/not spam", or "healthy/sick" to data points.
- LR uses an S-shaped curve that maps real-valued numbers to values between 0 and 1 → probabilistic values
- LR calculates the probability that a dependent variable will be categorized into a specific group, providing a probabilistic framework that can help with decision-making.

73

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION



74



PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

- A classification algorithm.
- A significant ML algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete variables.
- Can be used to classify the observations using **different types of data**
- Can easily determine the **most effective variables** used for the classification
- The Logistic Regression equation starts with a linear combination of input features, similar to linear regression and it takes the form:

$$y(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- where $x_1, x_2, \dots, x_n$ are the features, and $w_0, w_1, \dots, w_n$ are the weights or coefficients.
- Since for Logistic Regression, y can be between 0 and 1 only, equation can be re-written as:

$$\log\left(\frac{y}{1-y}\right) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- ⁷⁵ The dependent variable must be categorical in nature

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

- To convert the linear combination $y(x)$ into a probability (which must be between 0 and 1), logistic regression uses the **logistic function or sigmoid function**:

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

- The output of the sigmoid function is interpreted as the probability $P(y=1)$ of the positive class (e.g., "success").
- **Probability Prediction:** Logistic regression predicts the probability that the dependent variable y belongs to a particular class. For binary classification, this is usually the class labeled as 1.

$$P(y = 1|x) = \frac{1}{1 + e^{-(w_0 + w_1x_1 + \dots + w_nx_n)}}$$

- The model predicts the positive class if this probability is greater than a chosen threshold (commonly 0.5).
- **Decision Boundary:** It is the value of the input features where the probability $P(y=1)$ is 0.5. For a logistic regression model with two features, this boundary is a straight line → a straight line separates the 2 classes

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

Advantages:

- **Interpretable:** The coefficients in logistic regression can be interpreted as the impact of each feature on the probability of the positive class.
- **Simple and Efficient:** It is computationally efficient and performs well on linearly separable data.
- **Probabilistic Output:** Provides a probability score for predictions, which is useful for risk assessment.

Limitations:

- **Linear Decision Boundary:** Logistic regression assumes a linear relationship between the input features and the log-odds of the outcome, which may not capture complex patterns.
- **Sensitive to Outliers:** Like linear regression, logistic regression can be sensitive to outliers, which can skew the results.

PROBABILISTIC DISCRIMINATIVE MODEL - LOGISTIC REGRESSION

Types of Logistic Regression:

- **Binomial / binary Logistic Regression:** The categorical response has only 2 possible outcomes. Eg. True & False
- **Multinomial Logistic Regression:** The categorical response has 3 or more possible outcomes, without ordering. Eg. Cuisines → Indian, Chinese, Italian, Thai
- **Ordinal Logistic Regression:** The categorical response has 3 or more possible ordered outcomes; Eg. Low, Medium, High; Movie rating

Applications of Logistic Regression:

- Medical Diagnosis, Spam Detection, Customer Churn Prediction, Credit Scoring

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

- A probabilistic generative model → a type of model used in statistics and ML to represent how data is generated in terms of a probabilistic process.
- Provides a way to describe the joint probability distribution over both observed data and their corresponding labels (in a supervised learning context).
- Has a Generative Process
- Uses Joint Probability factored using Bayes' theorem
- Predict the class label for a new data point
- **Applications:** Text Classification, Image Recognition, Speech Recognition

79

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

- Consider a sample dataset that describes the weather conditions for playing a game of golf. Given the weather conditions, each tuple classifies the conditions as fit("Yes") or unfit("No") for playing golf.

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	FALSE	No
1	Rainy	Hot	High	TRUE	No
2	Overcast	Hot	High	FALSE	Yes
3	Sunny	Mild	High	FALSE	Yes
4	Sunny	Cool	Normal	FALSE	Yes
5	Sunny	Cool	Normal	TRUE	No
6	Overcast	Cool	Normal	TRUE	Yes
7	Rainy	Mild	High	FALSE	No
8	Rainy	Cool	Normal	FALSE	Yes
9	Sunny	Mild	Normal	FALSE	Yes
10	Rainy	Mild	Normal	TRUE	Yes
11	Overcast	Mild	High	TRUE	Yes
12	Overcast	Hot	Normal	FALSE	Yes
13	Sunny	Mild	High	TRUE	No

- The dataset is divided into two parts, namely, feature matrix and the response vector.
- Feature matrix contains all the vectors(rows) of dataset in which each vector consists of the value of dependent features. In the given dataset, features are 'Outlook', 'Temperature', 'Humidity' and 'Windy'.
- Response vector contains the value of class variable(prediction or output) for each row of feature matrix. In above dataset, the class variable name is 'Play golf'.

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

- The Naive Bayes classifier → a simple classifier model → a probabilistic ML model used for classification tasks.
- Based on Bayes' theorem
- Uses a frequency table of a dataset
- Assumes that the features (or predictors) are conditionally independent given the class label.
- This "naive" assumption of independence simplifies the computation and makes the model both fast and effective for many types of classification problems.
- According to Bayes theorem,
$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$
- It relates
 - the conditional probability of an event Y given evidence $X \rightarrow P(Y|X)$
 - the likelihood of X given Y , the prior probability of Y , and the marginal likelihood of $X \rightarrow P(X|Y)$, $P(Y)$ and $P(X)$

81

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

$P(Y|X) \rightarrow$ Posterior Probability: Probability of hypothesis Y on the observed event X .

$P(X|Y) \rightarrow$ Likelihood Probability: Probability of the evidence given that the probability of a hypothesis is true.

$P(Y) \rightarrow$ Prior Probability: Probability of hypothesis before observing the evidence.

$P(X) \rightarrow$ Marginal Probability: Probability of Evidence.

82

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Assumptions of Naive Bayes

- **Feature independence:** The features of the data are conditionally independent of each other, given the class label.
- **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.
- **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- **No missing data:** The data should not contain any missing values.

83

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

- In the context of classification, Y represents the class label, and X represents the feature vector.
- The key assumption in the Naive Bayes classifier $\rightarrow$ all features are independent of each other given the class label.
- Mathematically, this means: $P(X_1, X_2, \dots, X_n | Y) = P(X_1|Y) \cdot P(X_2|Y) \cdot \dots \cdot P(X_n|Y)$
- Hence
$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$
- This can be expressed as
$$P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1)P(x_2)\dots P(x_n)}$$
- As the denominator remains constant for a given input, we can remove that term. Hence

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

84

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

- The model assigns a class label Y to a new observation X by choosing the class that maximizes the posterior probability:

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

$$Y_{\text{pred}} = \arg \max_Y P(Y|X)$$

$$Y_{\text{pred}} = \arg \max_Y P(Y) \cdot \prod_{i=1}^n P(X_i|Y)$$

85

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES – EXAMPLE PROBLEM FOR CLASSIFICATION

Example-1:

Consider the given Dataset ,Apply Naive Baye's Algorithm and Predict that if a fruit has the following properties then which type of the fruit it is

Fruit = {Yellow , Sweet ,long}

Frequency Table:

Fruit	Yellow	Sweet	Long	Total
Mango	350	450	0	650
Banana	400	300	350	400
Others	50	100	50	150
Total	800	850	400	1200

86

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES – EXAMPLE PROBLEM FOR CLASSIFICATION

Solution:

- We must find $P(Y|X) = (P(X|Y) * P(Y)) / P(X)$ where
 - $Y \rightarrow$ Fruit (class label)
 - $X \rightarrow$ Yellow, Sweet, Long (features)
- For a new observation, find the probability for the 3 possible labels $\rightarrow$
 1. $P(Y=\text{Mango} | X = \text{Yellow, Sweet, Long})$,
 2. $P(Y=\text{Banana} | X = \text{Yellow, Sweet, Long})$,
 3. $P(Y=\text{Others} | X = \text{Yellow, Sweet, Long})$,
- Assign the class label Y with maximum probability to X

87

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

1. The fruit belongs to label 'Mango':

$$P(Y=\text{Mango} | X = \text{Yellow, Sweet, Long}) = \frac{P(X = \text{Yellow, Sweet, Long} | Y=\text{Mango}) * P(Y=\text{Mango})}{P(X = \text{Yellow, Sweet, Long})}$$

[Denominator is used for normalization; so optional during classification problem]

$$= P(X=\text{Yellow} | Y=\text{Mango}) * P(X=\text{Sweet} | Y=\text{Mango}) * P(X=\text{Long} | Y=\text{Mango}) * P(Y=\text{Mango})$$

$$P(Y = \text{Mango}) = 650 / 1200 = 0.5417$$

$$1. a) P(\text{Yellow} | \text{Mango}) = 350 / 650 = 0.5385 \rightarrow 1$$

$$1. b) P(\text{Sweet} | \text{Mango}) = 450 / 650 = 0.6923 \rightarrow 2$$

$$1. c) P(\text{Long} | \text{Mango}) = 0 / 650 = 0 \rightarrow 3$$

$$P(Y = \text{Mango} | X) = 0.5385 * 0.6923 * 0 * 0.5417 = 0$$

$$P(Y=\text{Mango} | X) = 0$$

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

2. The fruit belongs to label 'Banana':

$$P(Y=\text{Banana} \mid X = \text{Yellow, Sweet, Long}) = \frac{P(X = \text{Yellow, Sweet, Long} \mid Y=\text{Banana}) * P(Y=\text{Banana})}{P(X = \text{Yellow, Sweet, Long})}$$

$$= P(X=\text{Yellow} \mid Y=\text{Banana}) * P(X=\text{Sweet} \mid Y=\text{Banana}) * P(X=\text{Long} \mid Y=\text{Banana}) * P(Y=\text{Banana})$$

$$P(Y = \text{Banana}) = 400 / 1200 = 0.3333$$

$$1. a) P(\text{Yellow} \mid \text{Banana}) = 400/400 = 1 \rightarrow 1$$

$$1.b) P(\text{Sweet} \mid \text{Banana}) = 300 / 400 = 0.75 \rightarrow 2$$

$$1. c) P(\text{Long} \mid \text{Banana}) = 350 / 400 = 0.875 \rightarrow 3$$

$$P(Y=\text{Banana} \mid X) = 0.219$$

$$P(Y= \text{Banana} \mid X) = 1 * 0.75 * 0.875 * 0.3333 = 0.219$$

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

3. The fruit belongs to label 'Others':

$$P(Y=\text{Others} \mid X = \text{Yellow, Sweet, Long}) = \frac{P(X = \text{Yellow, Sweet, Long} \mid Y=\text{Others}) * P(Y=\text{Others})}{P(X = \text{Yellow, Sweet, Long})}$$

$$= P(X=\text{Yellow} \mid Y=\text{Others}) * P(X=\text{Sweet} \mid Y=\text{Others}) * P(X=\text{Long} \mid Y=\text{Others}) * P(Y=\text{Others})$$

$$P(Y = \text{Others}) = 150 / 1200 = 0.125$$

$$1. a) P(\text{Yellow} \mid \text{Others}) = 50/150 = 0.3333 \rightarrow 1$$

$$1.b) P(\text{Sweet} \mid \text{Others}) = 100 / 150 = 0.6667 \rightarrow 2$$

$$1. c) P(\text{Long} \mid \text{Others}) = 50 / 150 = 0.3333 \rightarrow 3$$

$$P(Y=\text{Others} \mid X) = 0.00926$$

$$P(Y= \text{Others} \mid X) = 0.3333 * 0.6667 * 0.3333 * 0.125 = 0.00926$$

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Consider the given Dataset ,Apply Naive Baye's Algorithm and Predict that if a fruit has the following properties then which type of the fruit it is
Fruit = {Yellow , Sweet ,long}

Frequency Table:

Fruit	Yellow	Sweet	Long	Total
Mango	350	450	0	650
Banana	400	300	350	400
Others	50	100	50	150
Total	800	850	400	1200

Solution:

So finally:

$P(\text{Mango} | X) = 0$, $P(\text{Banana} | X) = 0.219$ and $P(\text{Others} | X) = 0.00926$.

$P(Y) = \max(P(\text{Mango} | X) , P(\text{Banana} | X) , P(\text{Others} | X)) = P(\text{Banana} | X)$

We can conclude Fruit{Yellow, Sweet, Long} is Banana.

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Example-2:

Consider the following dataset. Apply Naïve Bayes classifier and predict if the weather is sunny, then the Player should play or not?

Solution steps:

1. Convert the given dataset into frequency table.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

1. Frequency table for the weather condition:

Weather	Yes	No	Total
Rainy	2	2	4
Sunny	3	2	5
Overcast	5	0	5
Total	10	4	14

93

2. Likelihood table for the weather condition:

Weather	Yes	No	Total	Likelihood
Rainy	2	2	4	$4/14 = 0.29$
Sunny	3	2	5	$5/14 = 0.35$
Overcast	5	0	5	$5/14 = 0.35$
Total	10	4	14	
Likelihood	$10/14 = 0.71$	$4/14 = 0.29$		

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

3. Using Bayes theorem to calculate the posterior probability:

Find $P(\text{Yes} | \text{Sunny})$ and $P(\text{No} | \text{Sunny})$

$$P(\text{Yes} | \text{Sunny}) = (P(\text{Sunny} | \text{Yes}) * P(\text{Yes})) / P(\text{Sunny})$$

$$= ((3 / 10) * (10/14)) / (5/14) = (0.3 \times 0.71) / 0.35 = 0.6$$

$$P(\text{Yes} | \text{Sunny}) = 0.6$$

$$P(\text{No} | \text{Sunny}) = (P(\text{Sunny} | \text{No}) * P(\text{No})) / P(\text{Sunny})$$

$$= ((2 / 4) * (4/14)) / (5/14) = (0.5 \times 0.29) / 0.35 = 0.4$$

$$P(\text{No} | \text{Sunny}) = 0.4$$

Since $P(\text{Yes} | \text{Sunny})$ has the maximum value, if the weather is sunny, the player can play the game.

94

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Overall Probabilities

P(Yes | Sunny, Rainy, Overcast) and P(No | Sunny, Rainy, Overcast)

$$P(\text{Yes} | \text{Sunny, Rainy, Overcast}) = \frac{P(\text{Sunny} | \text{Yes}) * P(\text{Rainy} | \text{Yes}) * P(\text{Overcast} | \text{Yes}) * P(\text{Yes})}{P(\text{Sunny, Rainy, Overcast})}$$

$$P(\text{No} | \text{Sunny, Rainy, Overcast}) = \frac{P(\text{Sunny} | \text{No}) * P(\text{Rainy} | \text{No}) * P(\text{Overcast} | \text{No}) * P(\text{No})}{P(\text{Sunny, Rainy, Overcast})}$$

95

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Advantages:

- One of the fastest and easy ML algorithms to predict a class of datasets.
- Can be used for Binary as well as Multi-class Classifications.
- Performs well in Multi-class predictions as compared to the other algorithms.
- The most popular choice for text classification problems.

Limitations:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features

PROBABILISTIC GENERATIVE MODEL – NAIVE BAYES

Types of Naive Bayes Model:

- **Gaussian:** The Gaussian model assumes that features follow a normal distribution → if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial:** The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed → primarily used for document classification problems (means classifying a particular document belongs to which category such as Sports, Politics, education, etc.) → the frequency of words used for the predictors.
- **Bernoulli:** The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables → like if a particular word is present or not in a document → famous model for document classification tasks.

Applications of Logistic Regression:

- Credit Scoring, Medical data classification, real-time predictions, Text classification (Spam filtering and Sentiment analysis).

97

DECISION TREE

- A simple representation for decision making and classifying samples.
- **Definition:** A decision tree is a flowchart-like structure used for decision-making. It visually represents possible outcomes of a series of choices, allowing users to weigh options based on costs, probabilities, and benefits.
- One of the most successful supervised learning techniques for classification
- In decision analysis, a decision tree can be used to **visually** and explicitly represent decisions and decision making – using a tree-like model
- One of the most widely used method for **inductive inference**.
- Robust to noisy data and capable of learning disjunctive expressions.

99

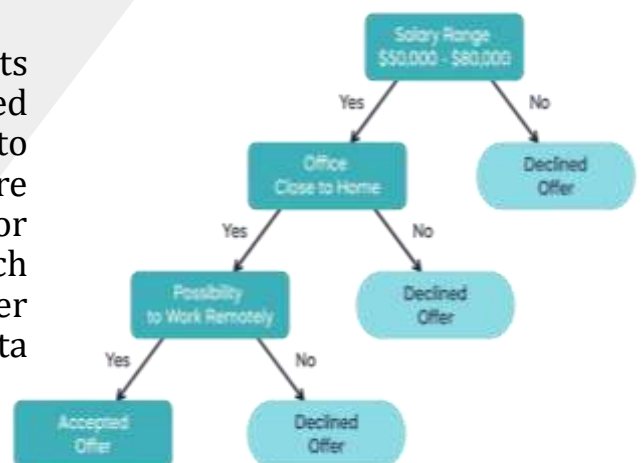
DECISION TREE

- Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance.
- Each node in the tree specifies a test of some attribute of the instance, and each branch descending from that node corresponds to one of the possible values for this attribute.
- An instance is classified by starting at the root node of the tree, testing the attribute specified by this node, then moving down the tree branch corresponding to the value of the attribute in the given example.
- This process is then repeated for the subtree rooted at the new node.
- Each path from the tree root to a leaf corresponds to a conjunction of attribute tests, and the tree itself corresponds to a disjunction of these conjunctions.

100

DECISION TREE

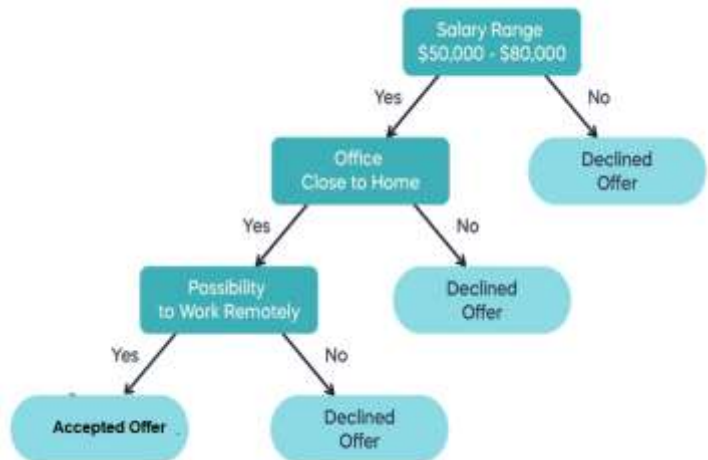
- Goal → to create a model that splits the data into distinct regions based on feature values, aiming to minimize some impurity measure (such as Gini index, entropy, or variance for regression) at each split. It does not explicitly consider margins or distances between data points.
- An example decision tree



101

DECISION TREE

- In graph theory, a tree is defined as an undirected graph in which any two vertices are connected by exactly one path.
- The nodes or vertices shown as ellipses are called the leaf nodes.
- All other nodes, except the root node, are called the internal nodes.



102

DECISION TREE

Day	Outlook	Humidity	Wind	PlayTennis
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

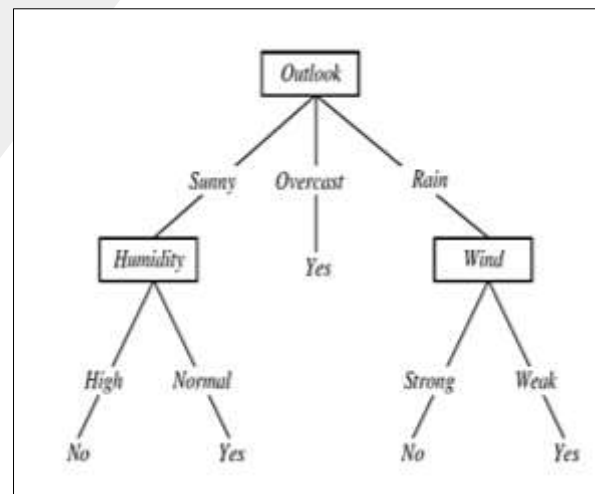
Conjunction($\wedge$) and Disjunction($\vee$): eg.

If Outlook=Sunny $\wedge$ Humidity=Normal

$\vee$ Outlook=Overcast

$\vee$ Outlook=Rain $\wedge$ Wind=Weak

then PlayTennis=Yes



A decision tree for the concept PlayTennis

DECISION TREE

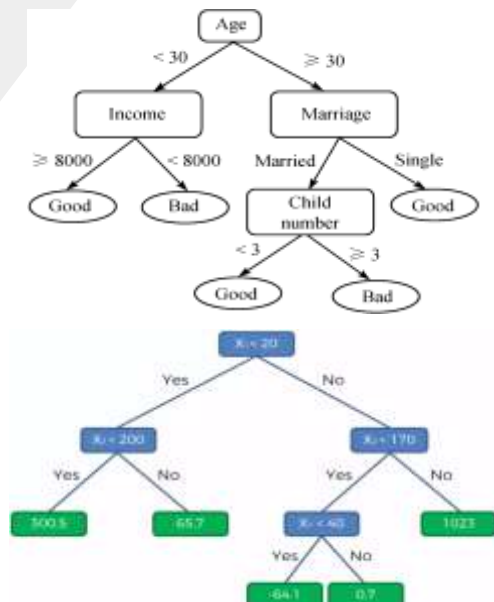
Types of decision trees

1. Classification trees

- Tree models where the target variable can take a discrete set of values are called classification trees.
- In these tree structures, leaves represent class labels and branches represent conjunctions of features that lead to those class labels.

2. Regression trees

- Decision trees where the target variable can take continuous values (real numbers) are called regression trees.
- Eg. House price, a patient's length of stay in a hospital,



DECISION TREE

Entropy

- The degree to which a subset of examples contains only a single class is known as purity, and any subset composed of only a single class is called a pure class.
- Entropy is a measure of “impurity” or heterogeneity in a dataset → For n classes, entropy ranges from 0 to $\log_2(n)$.

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

- Eg. Suppose S is a collection of 14 examples of some boolean concept, including 9 positive and 5 negative examples. Then the entropy of S relative to this Boolean classification is

$$\text{Entropy}(S) = (-9/14) \log_2(9/14) + (-5/14) \log_2(5/14) = 0.41 + 0.53 = 0.94$$

- Entropy = 0, if all members of S belong to the same class
- Entropy = 1, if equal number of positive and negative classes in S
- $0 < \text{Entropy} < 1$, if unequal number of positive and negative classes in S

DECISION TREE

Feature selection measures

- The most important problem in implementing the decision tree algorithm is **deciding which features are to be considered as the root node and at each level.**
- Several methods have been developed to assign numerical values to the various features such that the values reflect the relative importance of the various features → called the feature selection measures.
- Two of the popular feature selection measures are **information gain** and **Gini index.**
- Information gain** → expected reduction in entropy caused by partitioning the examples according to this attribute → for attribute A relative to the set S

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$

106

DECISION TREE

- Eg. Find the information gain for 'Wind'.

Wind	PlayTennis
Weak	No
Strong	No
Weak	Yes
Weak	Yes
Weak	Yes
Strong	No
Strong	Yes
Weak	No
Weak	Yes
Weak	Yes
Strong	Yes
Strong	Yes
Weak	Yes
Strong	No

- The dataset S contains 9 Yes and 5 No for the target variable 'Play Tennis' → S = [9 +, 5 -]
- So $\text{Entropy}(S) = (-9/14) \log_2(9/14) + (-5/14) \log_2(5/14) = 0.41 + 0.53 = 0.94$
- A → Wind: $\text{Values}(A) = \{\text{Weak}, \text{Strong}\}$
- Hence $S_{\text{Weak}} = [6 +, 2 -]$ and $S_{\text{Strong}} = [3 +, 3 -]$

$$\text{Entropy}(S) - \frac{|S_{\text{Weak}}|}{|S|} \text{Entropy}(S_{\text{Weak}}) - \frac{|S_{\text{Strong}}|}{|S|} \text{Entropy}(S_{\text{Strong}})$$

$$\text{Entropy}(S_{\text{Weak}}) = (-6/8) \log_2(6/8) + (-2/8) \log_2(2/8) = 0.811$$

$$\text{Entropy}(S_{\text{Strong}}) = (-3/6) \log_2(3/6) + (-3/6) \log_2(3/6) = 1$$

$$\text{Gain}(S, A) = 0.94 - \frac{8}{14} (0.811) - \frac{6}{14} (1) = 0.048$$

107

DECISION TREE

Other Feature selection measures

- **Gini index** → for the dataset S
$$\text{Gini}(S) = 1 - \sum_{i=1}^r p_i^2$$

- **Split information**

$$\text{SplitInformation}(S, A) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

- **Gain ratio**

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

108

DECISION TREE

Building A Classification Decision Tree

- To generate decision tree from the training dataset D:
- **Input:** Dataset D, Attribute list L, Attribute selection method M, Target_attribute T.
- **Algorithm:**
 1. Create a Root node for the tree
 2. If L is empty, return the single-node tree Root, with label = most common value of T in D
 3. If all Examples in D belong to same class C, return the single-node tree Root, with label = C
 3. Otherwise, identify feature that results in the greatest information gain. Set this feature to be the splitting criterion at the current node.
 - a. If the best information gain is 0, tag the current node as a leaf and return.
 4. Partition all instances according to attribute value of the best feature.
 5. Denote each partition as a child node of the current node.
 6. For each child node:
 - a. If the child node is “pure” (has instances from only one class) tag it as a leaf and return.
 - b. If not set the child node as the current node and recurse to step 2.

109

DECISION TREE

Regression Trees

- A regression problem → problem of determining a relation between one or more independent variables and an output variable which is a real continuous variable and then using the relation to predict the values of the dependent variables.
- Regression problems → in general related to prediction of numerical values of variables.
- Trees can also be used to make such predictions.
- Regression tree → A tree used for making predictions of numerical variables → also called as prediction tree.

Example problem

- Using the data in following table, construct a tree to predict the values of y.

x_1	1	3	4	6	10	15	2	7	16	0
x_2	12	23	21	10	27	23	35	12	27	17
y	10.1	15.3	11.5	13.9	17.8	23.1	12.7	43.0	17.6	14.9

DECISION TREE

Building A Regression Tree

- **Input:** A table with dependent and independent variables and their values
- **Algorithm:**
 1. Start with a single node containing all data points. Calculate the mean of the values of y (m_c) which are in the leaf c and sum of squares of errors in tree T (ST).
 2. If all the points in the node have the same value for all the independent variables, stop.
 3. Otherwise, search over all binary splits of all variables for the one which will reduce sum of squares of errors as much as possible
 - a. If the largest decrease in sum of squares would be less than some threshold, or one of the resulting nodes would contain less than q points, stop and if c is a node where we have stopped, then assign the value mean to the node.
 - b. Otherwise, take that split, creating two new nodes.
 4. At each new node, go back to Step 1.

DECISION TREE

Key remarks

1. On the elements of a classification tree

- The various elements in a classification tree are identified as below.
 - **Nodes** in the classification tree are identified by the **feature names** of the given data.
 - **Branches** in the tree are identified by the **values of features**.
 - The **leaf nodes** identified by are the **class labels**.

2. On the order in which the features are selected

- Any of the features can be chosen as the initial feature for splitting the data.
- The classification tree depends on the order in which the features are selected for partitioning the data.

3. Stopping criteria

- Real-world data → more example records → large number of features with many possible values → complex classification trees. So stop construction if
 - All (or nearly all) of the examples at the node have the same class.
 - There are no remaining features to distinguish among the examples.
 - The tree has grown to a predefined size limit.

112

DECISION TREE

Pruning a decision node → process of removing the subtree rooted at that node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with that node

Issues in learning decision trees

- Avoiding Overfitting the Data → using pre-pruning or post-pruning
- Reduced error pruning
- Rule post-pruning
- Incorporating Continuous-Valued Attributes
- Alternative Measures for Selecting Attributes → use Gain Ratio
- Handling Training Examples with Missing Attribute Values
- Handling Attributes with Differing Costs

113

DECISION TREE

Advantages of Decision Tree Algorithms

- Rules are simple and easy to understand
- Both nominal and numerical attributes can be handled.
- Capable of handling datasets with that may have errors and missing values.
- Self-explanatory trees.

Disadvantages of Decision Tree Algorithms

- Most of the algorithms require that the target attribute will have only discrete values
- Some problems are difficult to solve for classification
- Decision trees are less appropriate for estimation tasks where the goal is to predict the value of a continuous attribute

114

DECISION TREE

The following are the Decision tree algorithms used for classification and regression tasks in machine learning

1. ID3 (Iterative Dichotomiser 3) → uses Information Gain based on Entropy as splitting criterion
2. C4.5, C5.0 → uses Gain Ratio as splitting criterion → Handles both discrete and continuous data.
3. CART (Classification and Regression Trees) → uses Gini Index as splitting criterion → gives a binary tree
4. CHAID (Chi-square Automatic Interaction Detection)
5. MARS (Multivariate Adaptive Regression Splines)
6. Random Forest (Ensemble of Decision Trees)
7. Extra Trees (Extremely Randomized Trees)
8. XGBoost (Extreme Gradient Boosting)

115

DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

Example - Classification Tree

Consider the given training dataset. Build a decision tree for the data.

Here target attribute is 'PlayTennis' with labels 'Yes' and 'No'

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

116

DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

Step-1

Create the root node with a decision attribute

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$

Find the information gain for all attributes and choose the one with the highest information gain as the root node.

- $S = [9 +, 5 -]$
- $\text{Entropy}(S) = (-9/14) \log_2(9/14) + (-5/14) \log_2(5/14) = 0.41 + 0.53 = 0.94$
- $\text{Gain}(S, \text{Outlook}) = ?$
- $\text{Gain}(S, \text{Temperature}) = ?$
- $\text{Gain}(S, \text{Humidity}) = ?$
- $\text{Gain}(S, \text{Wind}) = ?$

$$\text{Gain}(S, \text{Outlook}) = 0.2465$$

$$\text{Gain}(S, \text{Temperature}) = 0.029$$

$$\text{Gain}(S, \text{Humidity}) = 0.151$$

$$\text{Gain}(S, \text{Wind}) = 0.048$$

- Decision attribute for the root node $\rightarrow$ the attribute with maximum Gain
- Hence OUTLOOK attribute is selected as the decision attribute for the root node.

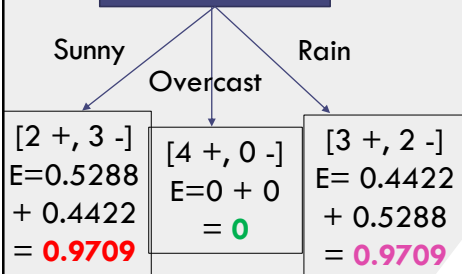
117

DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$

OUTLOOK



$$\text{Gain}(S, \text{Outlook}) = 0.94$$

$$- (5/14) \mathbf{0.9709}$$

$$- (4/14) \mathbf{0}$$

$$- (5/14) \mathbf{0.9709}$$

$$= 0.2465$$

Similarly

$$\text{Gain}(S, \text{Temperature}) = 0.029$$

$$\text{Gain}(S, \text{Humidity}) = 0.151$$

$$\text{Gain}(S, \text{Wind}) = 0.048$$

118

DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$

Since $\text{Entropy}(\text{Outlook}_{\text{Overcast}}) = 0$, Assign label 'Yes'

Now for $\text{Outlook}_{\text{Sunny}}$ and $\text{Outlook}_{\text{Rain}}$ identify the following gains

$$S_{\text{Sunny}} = D1, D2, D8, D9, D11$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Humidity}) = 0.97$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Temperature}) = 0.57$$

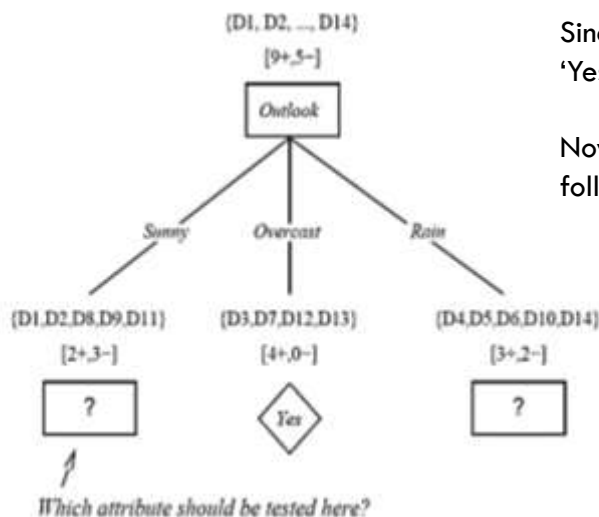
$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = 0.019$$

$$S_{\text{Rain}} = D4, D5, D6, D10, D14$$

$$\text{Gain}(S_{\text{Rain}}, \text{Humidity}) = 0.019$$

$$\text{Gain}(S_{\text{Rain}}, \text{Temperature}) = 0.019$$

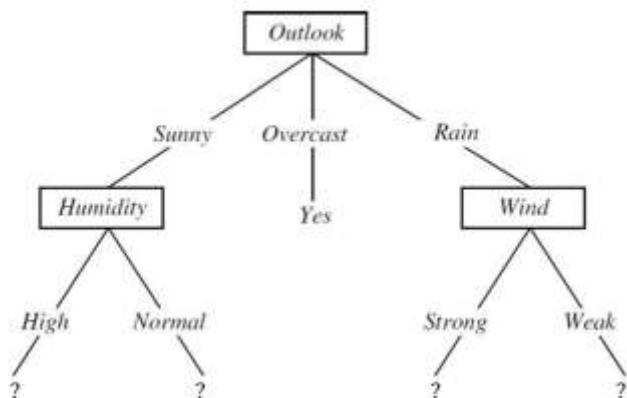
$$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = 0.97$$



DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$



The highest gains:

$$\text{Gain}(S_{\text{Sunny}}, \text{Humidity}) = 0.97$$

$$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = 0.97$$

Now calculate

$$\text{Gain}(S_{(\text{Sunny}, \text{High})}, \text{Temperature})$$

$$\text{Gain}(S_{(\text{Sunny}, \text{Normal})}, \text{Wind})$$

$$\text{Gain}(S_{(\text{Rain}, \text{Strong})}, \text{Humidity})$$

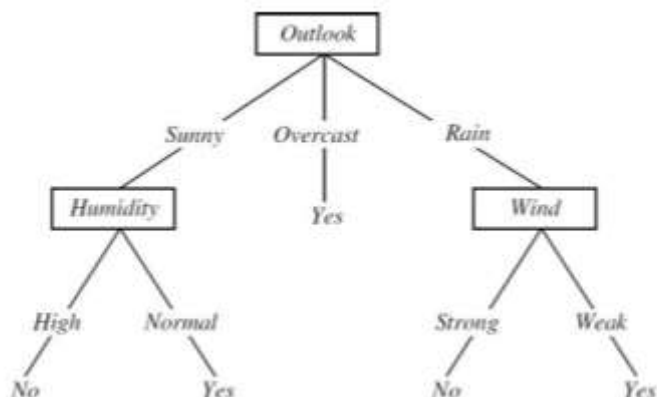
$$\text{Gain}(S_{(\text{Rain}, \text{Strong})}, \text{Temperature})$$

120

DECISION TREE

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$



The Entropy values for the above combination of features result in 0, leading to assigning class labels to the leaf nodes.

Hence we get the given tree

Note: Temperature is not considered at all here.

121

DECISION TREE

Example problem – Regression Tree

- Using the data in following table, construct a tree to predict the values of y .

T1

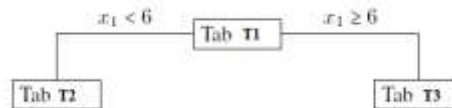
x_1	1	3	4	6	10	15	2	7	16	0
x_2	12	23	21	10	27	23	35	12	27	17
y	10.1	15.3	11.5	13.9	17.8	23.1	12.7	43.0	17.6	14.9

- Step 1:** Mean of $x_1 = 6$ So split the values of x_1 into two sets: $x_1 < 6$ and $x_1 \geq 6$

T2	x_1	1	3	4	2	0
	x_2	12	23	21	35	17
	y	10.1	15.3	11.5	12.7	14.9

T3	x_1	6	10	15	7	16
	x_2	10	27	23	12	27
	y	13.9	17.8	23.1	43.0	17.6

- The initial regression tree:



122

DECISION TREE

Example problem

- Step 2:**
- Consider T2 → Mean of $x_2 = 21$ → So split the values of x_2 into two sets: $x_2 < 21$ and $x_2 \geq 21$ → T4, T5

T4

x_1	1	0
x_2	12	17
y	10.1	14.9

T5

x_1	3	4	2
x_2	23	21	35
y	15.3	11.5	12.7

- Similarly consider T3 - Mean of $x_1 = 20$ → So split the values of x_2 into two sets: $x_2 < 20$ and $x_2 \geq 20$ → T6, T7

T6	x_1	6	7
	x_2	10	12
	y	13.9	43.0

T7	x_1	10	15	16
	x_2	27	23	27
	y	17.8	23.1	17.6

123

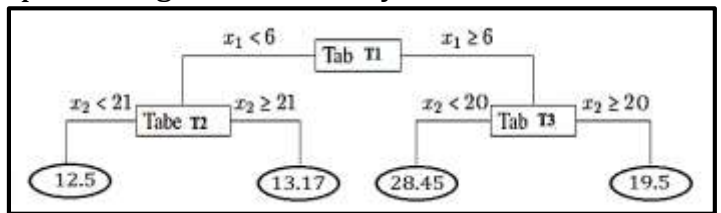
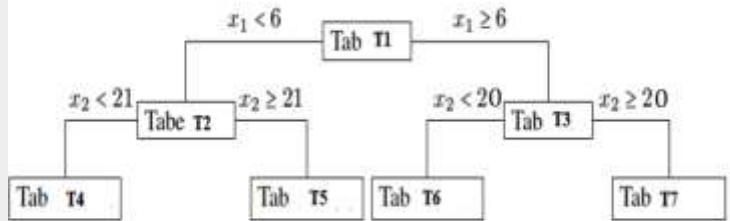
DECISION TREE

Example problem

- Hence the regression tree

Last Step:

- No more variables to split
- Hence form the leaf nodes by computing the average of the values in the corresponding tables T4, T5, T6 and T7. for eg. From T4, $\text{mean}(10.1, 14.9)=12.5$
- The final raw regression tree for predicting the values of y based on the data in table T1 is below:



124

DECISION TREE

CART Algorithm

- Decision trees can be used to create a model that predicts the value of a target (or dependent variable) based on the values of several input or independent variables.
- The CART, or Classification And Regression Trees methodology, was introduced in 1984 by Leo Breiman, Jerome Friedman, Richard Olshen and Charles Stone as an umbrella term to refer to the following types of decision trees:
 - Classification trees where the target variable is categorical and the tree is used to identify the "class" within which a target variable would likely fall into.
 - Regression trees where the target variable is continuous and tree is used to predict it's value.
- The main elements of CART are:
 - Rules for splitting data at a node based on the value of one variable
 - Stopping rules for deciding when a branch is terminal and can be split no more
 - A prediction for the target variable in each terminal node.

126

RANDOM FORESTS

- A supervised machine learning algorithm made up of decision trees.
- Used for both classification and regression - for example, classifying whether an email is "spam" or "not spam".
- Based on the concept of ensemble learning → a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.
- "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."
- Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- The following steps explain the working Random Forest Algorithm:

126

RANDOM FORESTS

How does Random Forest algorithm work?

- Random Forest works in two-phase
 1. Creating the random forest by combining N decision trees
 2. Making predictions for each tree created in the first phase.

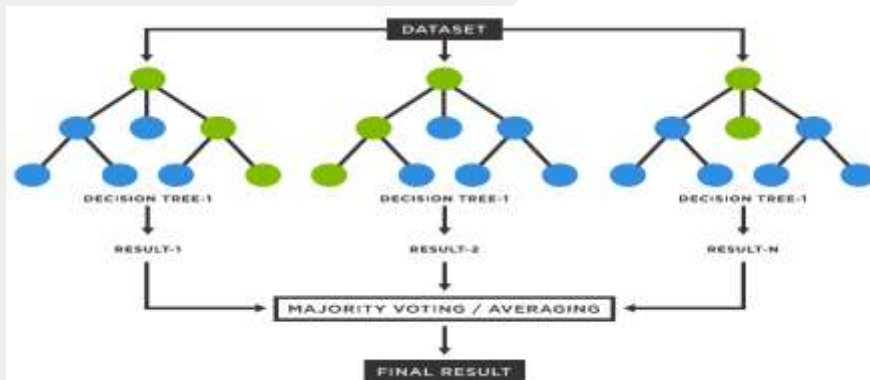


127

RANDOM FORESTS

How does Random Forest algorithm work?

- Random Forest works in two-phase
 1. Creating the random forest by combining N decision trees
 2. Making predictions from any / all tree created in the first phase.



128

RANDOM FORESTS

How does Random Forest algorithm work?

Step 1: Random Sampling (Bootstrap Aggregation / Bagging)

- From the training dataset, select random samples **with replacement** (called bootstrap samples).
- Each decision tree will be trained on a different bootstrap sample.

Step 2: Build Decision Trees with Feature Randomness

- When splitting a node in a tree, instead of considering **all features**, Random Forest selects a **random subset of features**.
- The best split is chosen only among this subset.
- This adds extra randomness, making trees less correlated.

Step 3: Repeat for N Trees

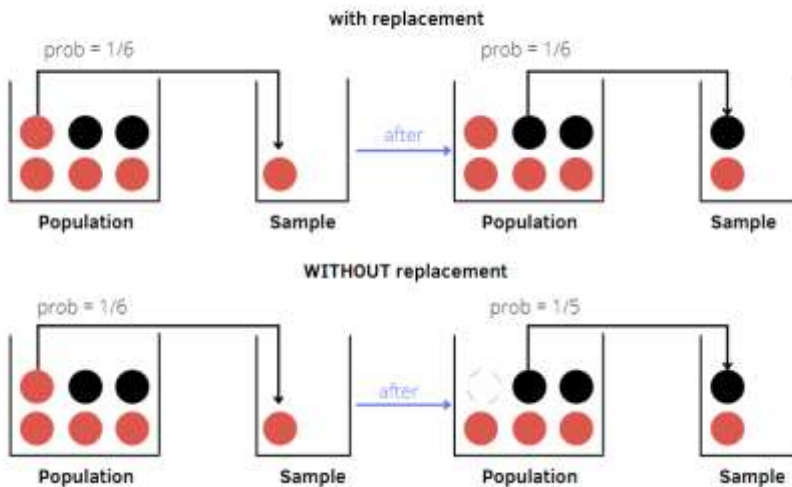
- Repeat Steps 1 & 2 to build **N decision trees** (where N is predefined, e.g., 100).

Step 4: Aggregation (Ensemble Prediction)

- For **classification**: each tree votes for a class → the majority vote is the final output.
- For **regression**: the predictions from all trees are averaged to get the final output.

129

RANDOM FORESTS



130

RANDOM FORESTS

Implementation Steps:

1. Data Pre-processing step
2. Fitting the Random Forest algorithm to the Training set
3. Predicting the test result
4. Test accuracy of the result (Creation of Confusion matrix)
5. Visualizing the test set result.

Applications of Random Forest

- Banking: Banking sector mostly uses this algorithm for the identification of loan risk.
- Medicine: With the help of this algorithm, disease trends and risks of the disease can be identified.
- Land Use: We can identify the areas of similar land use by this algorithm.
- Marketing: Marketing trends can be identified using this algorithm.

131

RANDOM FORESTS

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

132

CONFUSION MATRIX

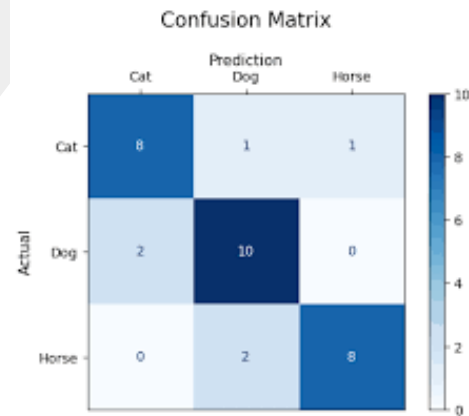
- **Definition:** A confusion matrix is a table used to evaluate the performance of a classification model in machine learning. It provides a detailed breakdown of how well the model's predictions align with the actual values in a dataset.
- For a binary classification problem (e.g., predicting "Positive" or "Negative"), the confusion matrix is typically a 2x2 table with four key components:
 - True Positive (TP): The model correctly predicted the positive class.
 - True Negative (TN): The model correctly predicted the negative class.
 - False Positive (FP): The model incorrectly predicted the positive class when the actual class was negative (Type I error).
 - False Negative (FN): The model incorrectly predicted the negative class when the actual class was positive (Type II error).
- In multiclass classification, the confusion matrix expands to an $N \times N$ table, where N is the number of classes, providing a similar breakdown of predictions across all categories.

133

CONFUSION MATRIX

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{TP + FN}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{TN + FP}$
		Precision $\frac{TP}{TP + FP}$	Negative Predictive Value $\frac{TN}{TN + FN}$	Accuracy $\frac{TP + TN}{TP + TN + FP + FN}$

Binary classification problem



Multiclass classification

134

CONFUSION MATRIX

- The rows of the confusion matrix typically represent the actual classes, while the columns represent the predicted classes. By examining the values in each cell, one can understand:
 - **Correct Predictions:** TP and TN represent instances where the model made accurate predictions.
 - **Errors:** FP and FN represent instances where the model made incorrect predictions. The type of error (false positive vs. false negative) can have different implications depending on the problem domain.
- The confusion matrix is fundamental for calculating various performance metrics for classification models, including:
 - Accuracy: $(TP + TN) / (TP + TN + FP + FN)$
 - Precision: $TP / (TP + FP)$
 - Recall (Sensitivity): $TP / (TP + FN)$
 - F1-Score: The harmonic mean of precision and recall.

135